FamPlan Demo Outline and Lab 4 and 5 Checklist

Banana Five-O · Draft, 31 October 2026 · Checked against main with the commute update, a clean test run and the live site, which does not have the update yet

The live demo fits 20 minutes: the 14 minutes of demo the team agreed, about 5 minutes on design and traceability, and a short close (Lab 5 §3.1). The video script is ready to record this weekend.

Sort out these first:

- **OneMap login, before recording:** a spec note says the team's renewed token expires on Sunday 1 Nov at 18:04 Singapore time. Confirm the live site uses account login, or record before then.
- **Commute update:** both of Ilham's commute items are fixed and merged to main, and both scripts now show them. The live site does not have them until Edward applies the database migration and deploys. Re-shoot the live-site screenshots after that.
- **Administrator login:** Edward's part needs a live administrator account, and whether one exists is still unconfirmed.
- **AI report:** this Lab 4 deliverable is not in the repo on any branch.
- **Old script:** `FamPlan_Demo_Script.docx` still describes an older build, so several of its lines are wrong. See Claims to use and drop.

About the screenshots: "Live site" shots come from famplan-amber.vercel.app on 31 Oct, before the commute update, so they still show Transit and Drive until re-shot. Screens that need a login come from a local test copy with test accounts and real public data, so replace them once a live account is ready. The architecture and design layer figures come from SRS v1.6 and the other diagrams from the Lab 3 and Lab 4 documents, so swap in Ilham's re-renders when they are done.

Run of show

Time	Presenter	Part	Use cases	On screen
0:00–2:00	**Apollo**	Problem, value, actors, stack, register and log in	UC-1.1, UC-1.2	Landing, use case diagram, Register
2:00–5:00	**Raph**	Family priorities and search	UC-2.1, UC-3.1, UC-3.2	Preferences, Explore
5:00–9:00	**Ilham**	Selected location, radius, map, commute	UC-4.1, UC-4.2, UC-5.1, UC-5.2	Profile, Commute
9:00–12:00	**Athena**	Score, comparison, shortlist	UC-6.1, UC-6.2, UC-6.3	Profile, Compare, Shortlist
12:00–14:00	**Edward**	Data monitoring	UC-7.1	Data sync dashboard
14:00–16:30	**Edward**	SE practices, system design, future upgrades	None	Slides: SRS §8.2, §9.4
16:30–19:00	**Ilham**	Traceability for one use case	UC-6.1	SRS, VS Code, terminal, test document
19:00–19:30	**Apollo**	Close	None	Landing
19:30–20:00	**All**	Buffer, then Q&A	None	None

The chat table stops at Edward with 14 minutes, so this outline gives the design slot to Edward and traceability to Ilham, as in the old script. Change these if the team split them differently.

UC-1.3 to UC-1.5 (reset password, manage account, delete account) are mentioned, not shown. The live site has no email service configured, so a password reset cannot complete there.

If the TA holds us to the 15 minutes in Lab 4 §2.6, start already signed in, compare two neighbourhoods instead of three, and give design and traceability 90 seconds each.

Live demo script

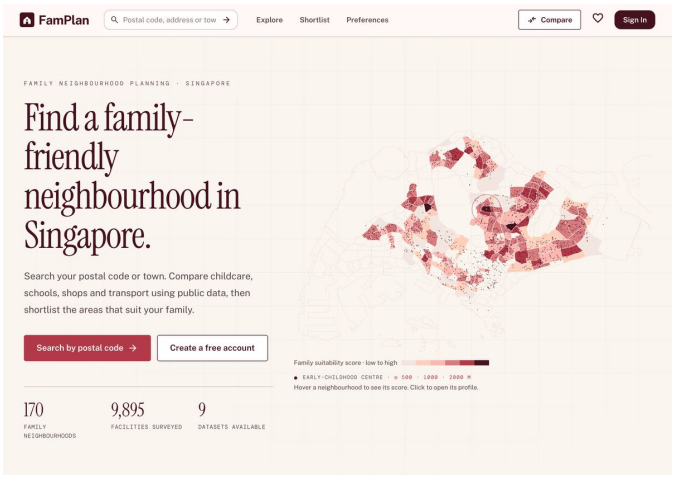
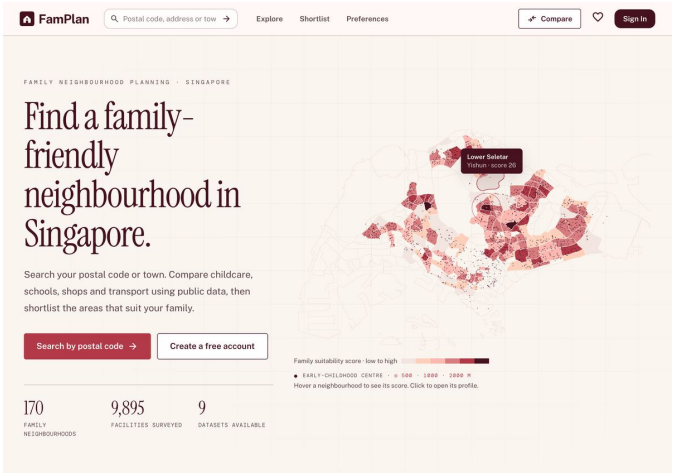
Every part follows one family, the Tans: a toddler, a child starting Primary 1, a parent working at one-north, and grandparents in Bedok. Each presenter shows one step of their decision and adds one engineering or design point.

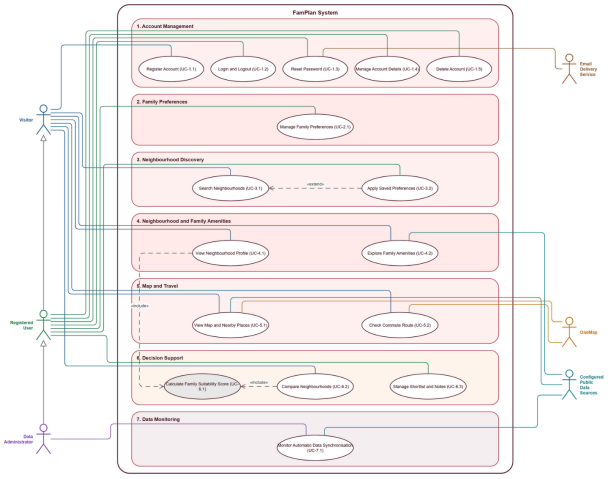
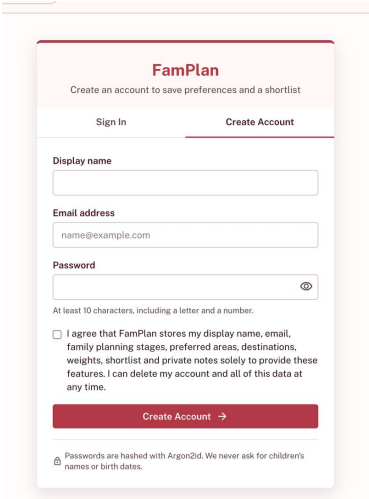
Land these six star features. Each pairs value for the family with an engineering or design point.

Star feature	Shown by	Why it matters, and the point to make
A score that explains itself	**Athena**	Families see why a neighbourhood fits, not just a number. Design: 8 weighted criteria, missing data disclosed, and one versioned configuration.
Exact address search	**Raph, Ilham**	A family is choosing a home, not a zone. Design: OneMap resolves the address, and the radius, counts and score all recalculate from that coordinate.
Commute for the whole family	**Raph**	Each destination has its own travel mode, and every destination moves the ranking. Design: a simple average that is easy to explain.
Data that refreshes itself	**Edward**	9 public datasets update daily with no uploads. Engineering: validated snapshots stay live when a provider fails, as happened with Overpass.
Live routes that stay honest	**Ilham**	OneMap routes in 4 modes, and the app says when step-free data or routing is unavailable.
Built to scale and extend	**Apollo, Edward**	A new source is one adapter. 283 automated tests. In a local load test, one server handled 1,000 simulated users with no failed requests.

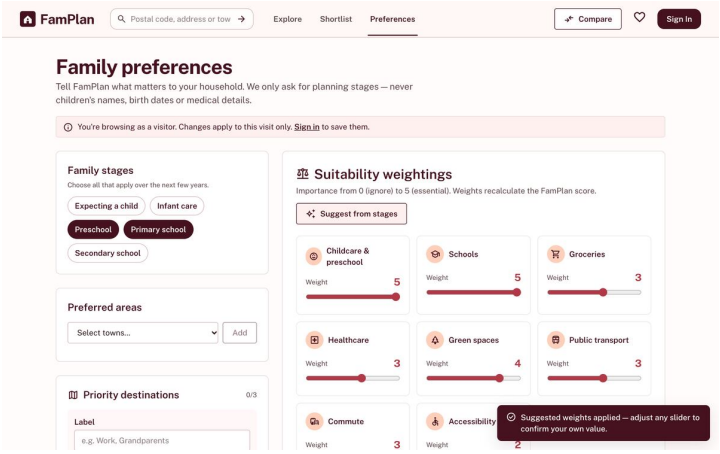
This covers the teacher's notes: value proposition in Apollo's opening, architecture and stack in Apollo's part near the start, engineering points in each "Point to add", and design choices on exact search and the commute average.

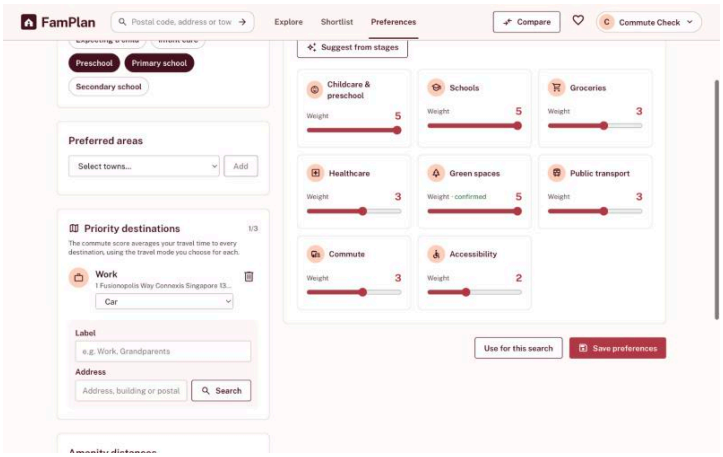
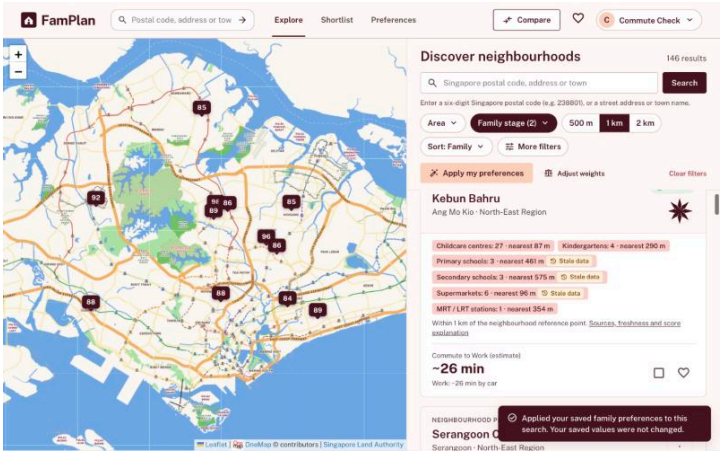
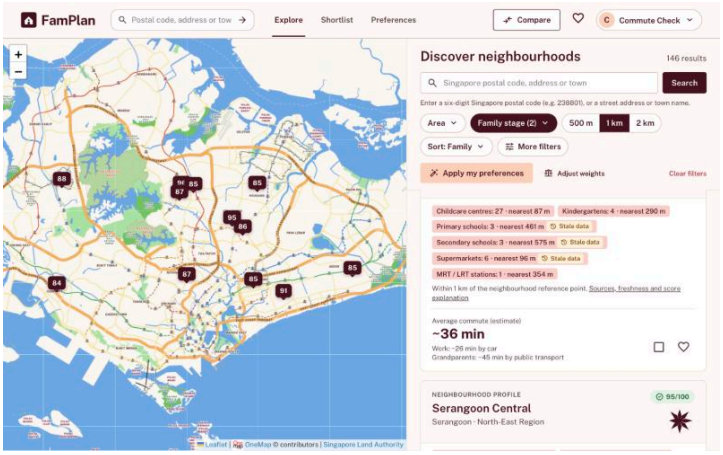
Apollo, 0:00–2:00: the problem and who it serves

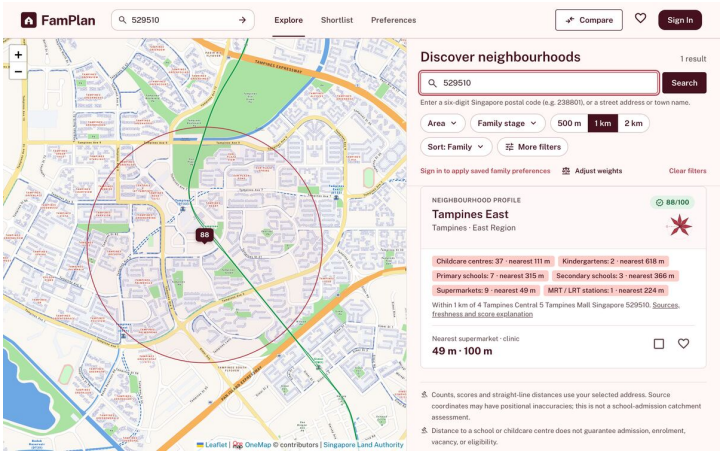
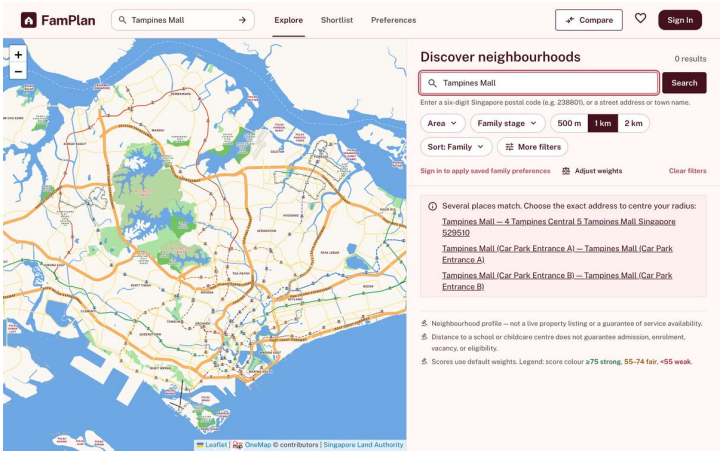
Script	Visual reference
SHOW Landing page. Scroll slightly so the counters run. **SAY** FamPlan is for couples planning a family in Singapore. Today they check childcare, schools, clinics, MRT and commute across separate sites. FamPlan puts them in one place, with a score for their own family that they can explain.	 Live site
SHOW Hover a neighbourhood on the landing map. **SAY** This is real public data, not sample data. **POINT TO ADD** 170 neighbourhoods, 9,895 facilities and 9 datasets. We list residential subzones with at least three early-childhood centres.	 Live site, with a neighbourhood hover label

Script	Visual reference
SHOW Use case diagram, then the architecture figure with one line on the stack. **SAY** There are three actors: visitor, registered user and data administrator. FamPlan is built with React and TypeScript, an Express API and PostgreSQL through Prisma, with Leaflet and OneMap for maps. **POINT TO ADD** 16 use cases. Hosted on Vercel in Singapore. In a local load test, one server handled 1,000 simulated users browsing with no failed requests.	 Use Case Descriptions v1.0, Figure 1
SHOW Register, tick consent, log in. **SAY** Visitors can search, view, compare, check commutes and try weights for one visit. Registered users also save preferences, a shortlist and private notes. **POINT TO ADD** Passwords need at least 10 characters, with a letter and a number, and are hashed with Argon2id. Sessions are HTTP-only.	 Live site

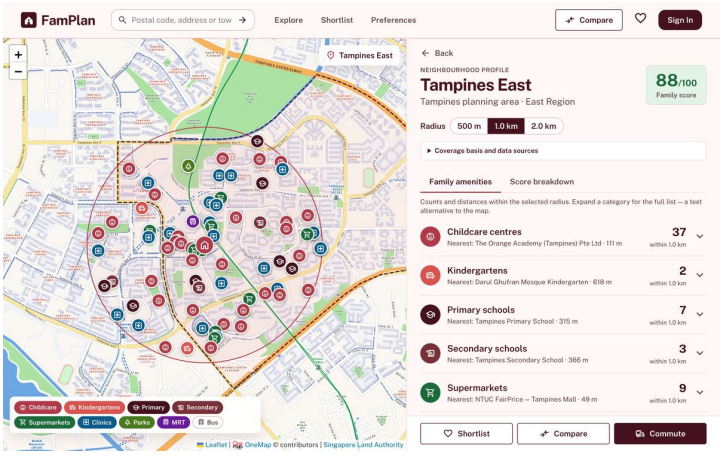
Raph, 2:00–5:00: family priorities and search

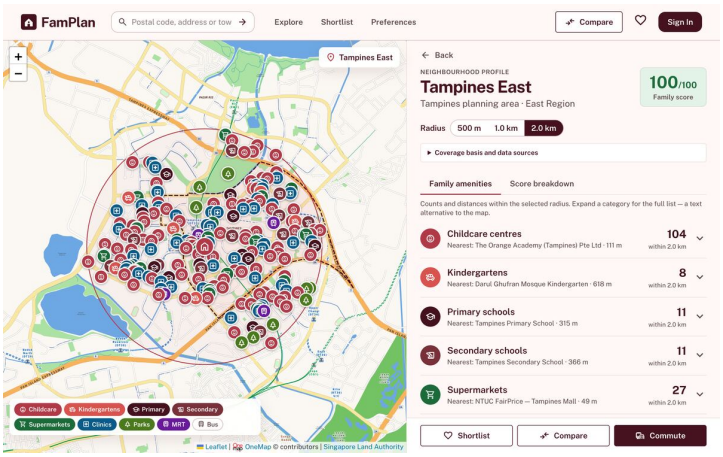
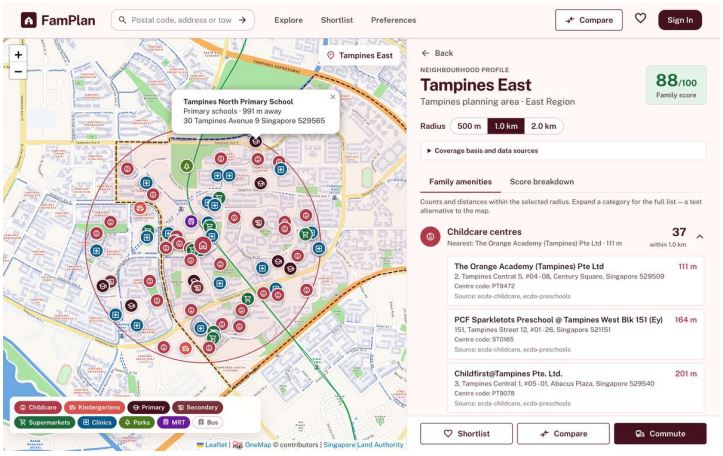
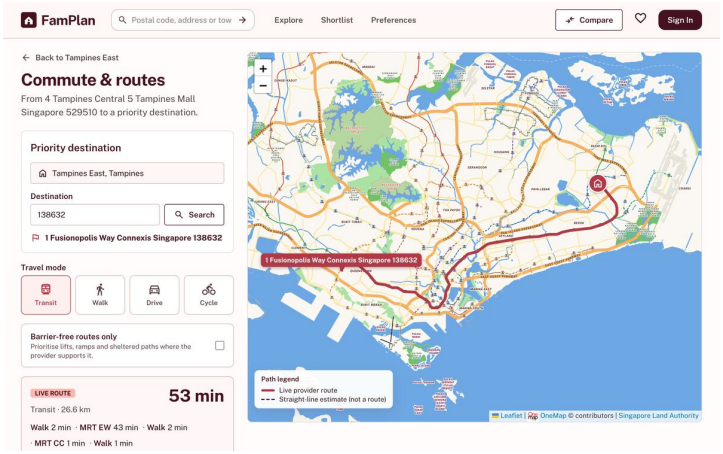
Script	Visual reference
SHOW Preferences: tick Preschool and Primary school, click Suggest from stages. **SAY** Weights are this family's priorities: 0 means ignore, 5 means essential. FamPlan suggests a starting point from their stages. **POINT TO ADD** We ask for planning stages only, never children's names, birth dates or medical details.	 Live site, visitor view

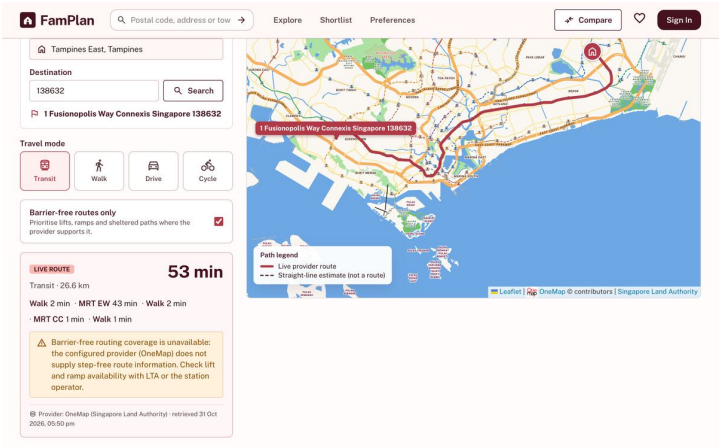
Script	Visual reference
SHOW Move Green spaces to 5. Add the label Work with postal code 138632, choose Car as its travel mode, then click Save preferences. **SAY** Moving a weight marks it as confirmed, so later suggestions keep it. Each destination has its own travel mode, because one parent might drive while the other takes the MRT. **POINT TO ADD** One validation schema is shared by client and server, so they cannot disagree.	 Local test copy, signed in
SHOW Explore: Apply my preferences. **SAY** The ranking now follows this family's priorities, and the list narrows to their family stages. **POINT TO ADD** Top three on the local copy: Kebun Bahru 98, Serangoon Central 96, Jelebu 92 (31 Oct; recheck before the demo).	 Local test copy, signed in
SHOW Preferences: add the label Grandparents with Bedok MRT, keep Public transport, then click Save preferences. Explore: Apply my preferences. **SAY** Every destination counts now. The commute score averages both trips, each by its own travel mode, so the ranking changes. **POINT TO ADD** Katong replaces Jelebu in the top three on the local copy (31 Oct; recheck). Design choice: a simple average, because it needs no extra input and is easy to explain in the breakdown.	 Local test copy, signed in

Script	Visual reference
SHOW Search postal code 529510. **SAY** A family thinks in addresses, not planning zones, so a postal code goes straight to the exact place. **POINT TO ADD** Design choice: we added exact search because a town centre can be far from the home being considered. How: OneMap turns the postal code into a coordinate, and FamPlan recalculates the radius, counts and score from it.	 Live site
SHOW Search the place name Tampines Mall, pick the first of the 3 matches. **SAY** Several places match, so FamPlan asks instead of guessing. **POINT TO ADD** OneMap resolves the address. A provider outage is never reported as "no such place".	 Live site

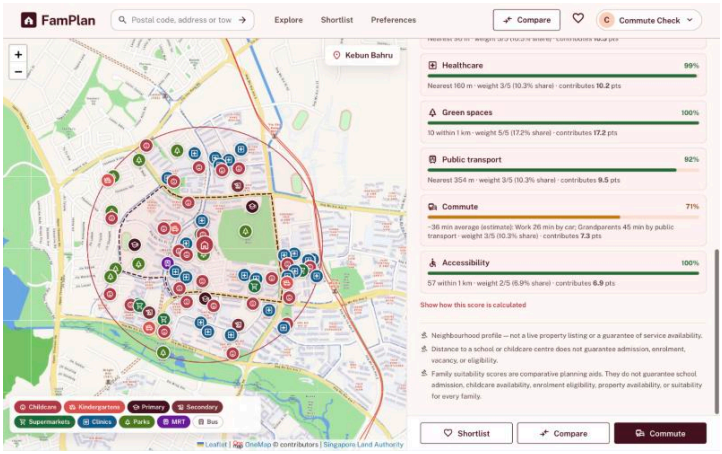
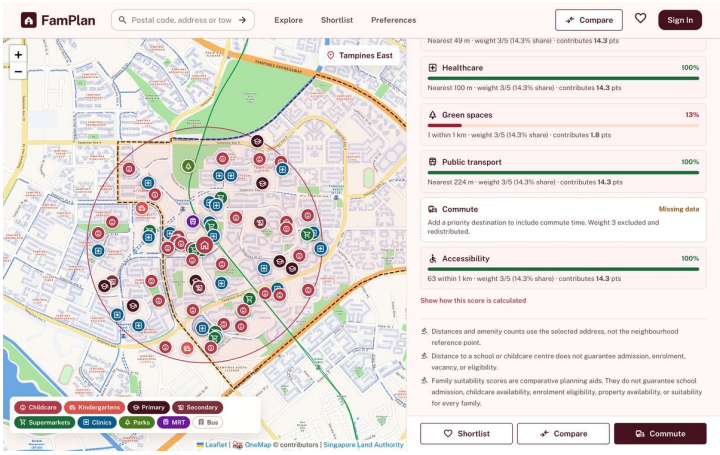
Ilham, 5:00–9:00: the selected location and the commute

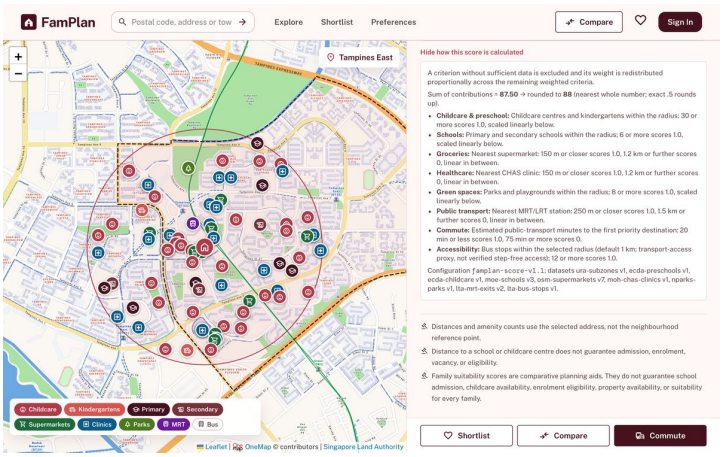
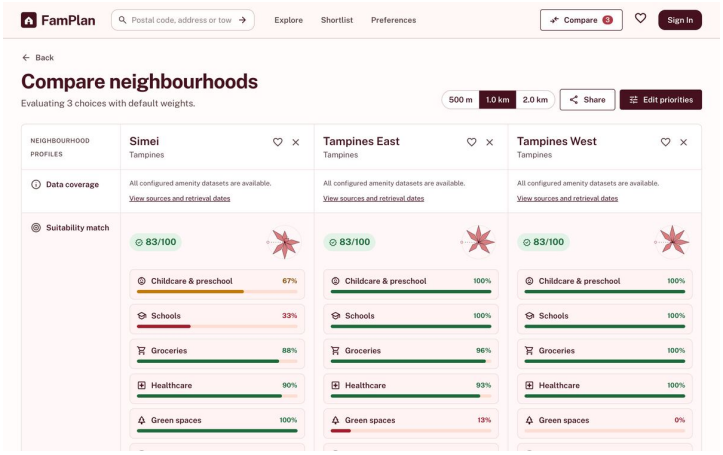
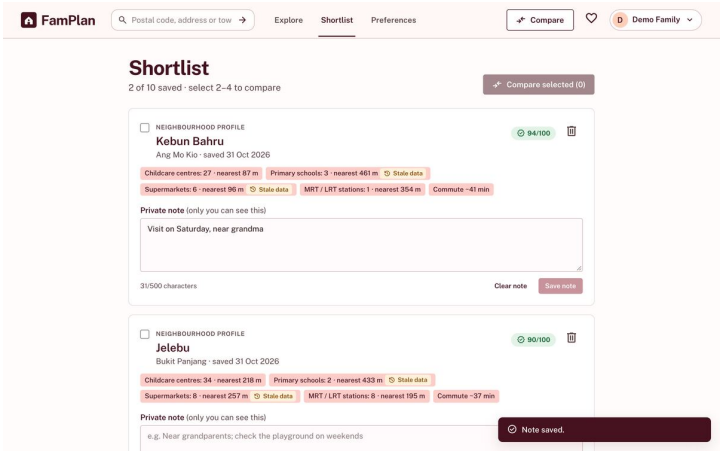
Script	Visual reference
SHOW Open the Tampines East result. **SAY** Radius calculations start from the selected address, not the centre of the zone. **POINT TO ADD** Tampines East scored 83 from its reference point and 88 from this address on default weights (31 Oct; recheck before the demo).	 Live site, 1 km

Script	Visual reference
SHOW Radius 500 m, 1 km, 2 km. **SAY** The map circle and the counts update together. **POINT TO ADD** Inclusion uses unrounded great-circle metres; rounding is for display only. It is not a walking distance or a school catchment.	 Live site, 2 km. Childcare count on 31 Oct: 13 at 500 m, 37 at 1 km, 104 at 2 km.
SHOW Toggle map layers, click a marker, expand Childcare centres. **SAY** Each entry in the list names its data source, and the list is an accessible alternative to the map. **POINT TO ADD** Source and retrieval dates are under Coverage basis and data sources. 9 amenity categories.	 Live site, marker popup and expanded list
SHOW Commute: the saved destination Work opens on Car. Then click Public transport, Walk and Cycle. **SAY** These are live routes from OneMap. The score uses the mode saved in preferences, so changing it here is only a preview. **POINT TO ADD** If routing is unavailable, the page shows a labelled estimate instead of failing.	 Live site before the update, Live route badge at the bottom left. Re-shoot after deploying.

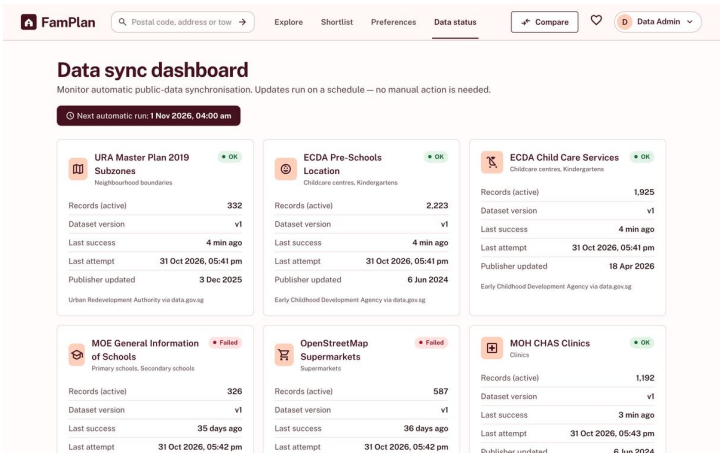
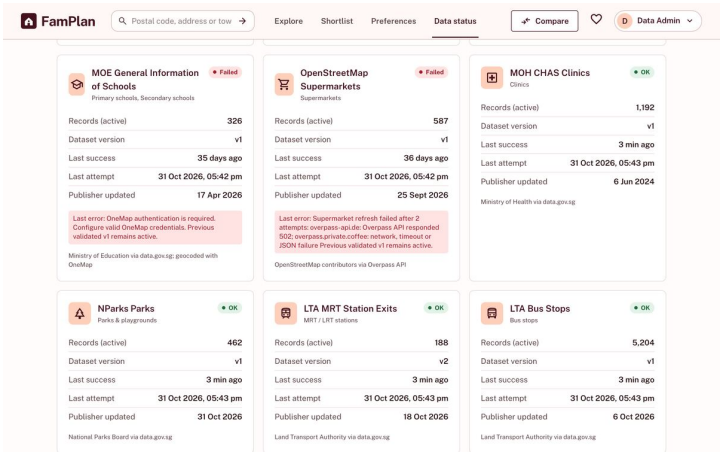
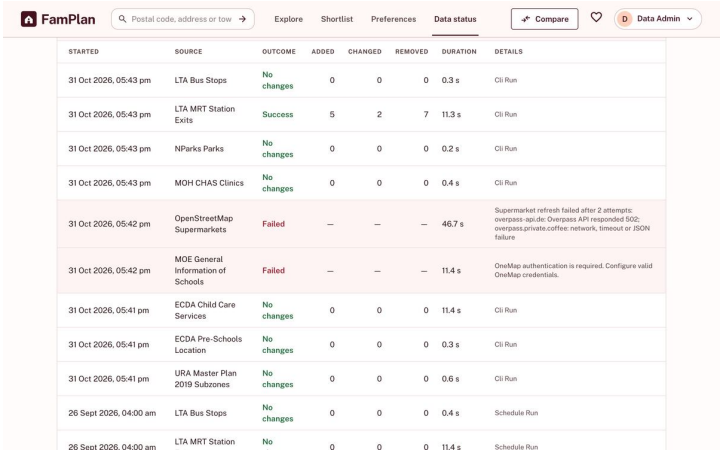
Script	Visual reference
SHOW Tick Barrier-free routes only. **SAY** OneMap supplies no step-free data, so FamPlan says that instead of guessing. **POINT TO ADD** The accessibility score is a bus-stop proxy, not verified step-free access.	 Live site

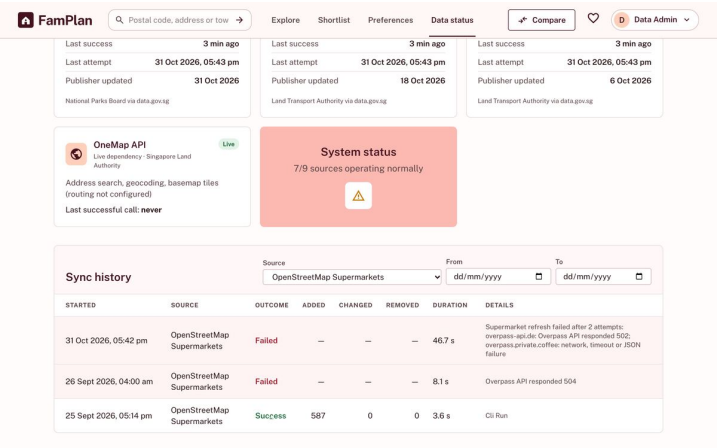
Athena, 9:00–12:00: score, comparison and shortlist

Script	Visual reference
SHOW Score breakdown tab on the profile. **SAY** The score compares neighbourhoods for this family. It does not promise a school place or a childcare vacancy. **POINT TO ADD** Each criterion shows its raw value, normalised value, weight and contribution. The Commute row lists each trip and its travel mode.	 Local test copy, signed in, two destinations
SHOW Switch to a signed-out window on the same profile and point at the Commute row. Once a destination is saved, commute is no longer missing, so this needs the signed-out view. **SAY** Missing data is disclosed, not hidden. **POINT TO ADD** Commute is excluded until a destination is set, and its weight is redistributed.	 Live site, signed out

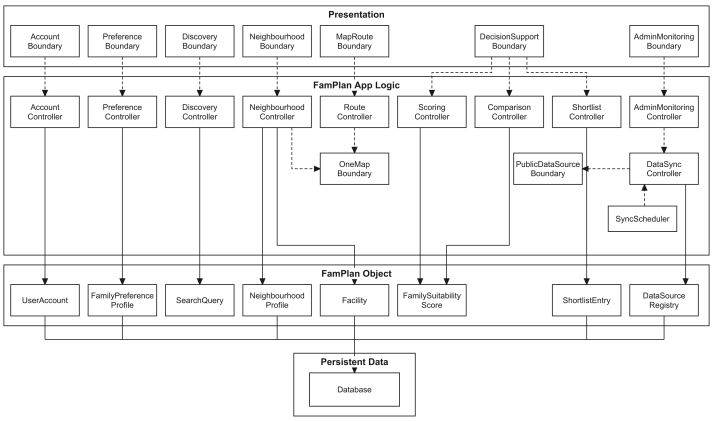
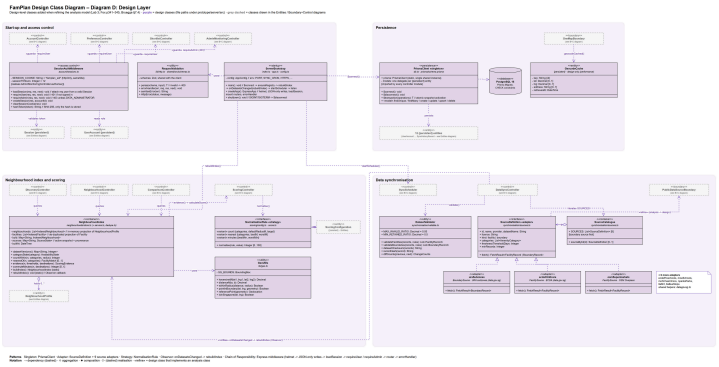
Script	Visual reference
SHOW Click Show how this score is calculated. **SAY** The same inputs always give the same score. **POINT TO ADD** The rules live in one versioned configuration, famplan-score-v1.2, recorded with the dataset versions.	 Live site
SHOW Add 3 neighbourhoods to Compare. **SAY** Compare puts 2 to 4 neighbourhoods side by side. **POINT TO ADD** Saved comparisons and shortlists work at neighbourhood level, so a score can differ from the exact-address view. Say this out loud.	 Live site, default weights
SHOW Save two, open Shortlist, add a note. **SAY** The shortlist holds up to 10 neighbourhoods, each with a private note of up to 500 characters. **POINT TO ADD** The limit is enforced in both the API and PostgreSQL.	 Local test copy, signed in

Edward, 12:00–14:00: keeping the data trustworthy

Script	Visual reference
SHOW Data sync dashboard, signed in as administrator. **SAY** The administrator never uploads data by hand. All 9 sources refresh automatically every day. **POINT TO ADD** Each source has its own overnight window, and a database lease prevents duplicate runs.	 Local test copy. The live dashboard will differ, so replace this shot.
SHOW Source status cards. **SAY** Each source shows its status, last success and record count. **POINT TO ADD** Delayed after 26 hours, Stale after 31 days.	 Local test copy, 31 Oct. These failures are real: both Overpass servers failed during our refresh, and MOE failed because the local copy has no OneMap login.
SHOW Sync history. **SAY** Every run is logged with its outcome and the records added, changed and removed. **POINT TO ADD** New data must pass validation before it becomes active.	 Local test copy

Script	Visual reference
SHOW Filter history to OpenStreetMap Supermarkets. **SAY** When a provider fails, the previous validated snapshot stays live, so families never see an empty map. **POINT TO ADD** Real case: Overpass returned HTTP 502. The snapshot held, and we added failover to one mirror.	 Local test copy, filtered to supermarkets

Edward, 14:00–16:30: SE practices, design and upgrades

Script	Visual reference
SHOW Slide: SRS §8.2 architecture, enlarged. **SAY** React boundary pages, Express controllers, domain entities, and PostgreSQL through Prisma.	 SRS v1.6 §8.2. Full size in the appendix.
SHOW Slide: SRS §9.4 design layer. **SAY** Each data source sits behind one Adapter. Scoring rules are a Strategy, selected from the versioned configuration.	 SRS v1.6 §9.4. Full size in the appendix.

SHOW
Slide: practices.
SAY
Feature branches and pull requests, database migrations, automated tests, and a design and verification note per change in `prototype/specs`.
POINT TO ADD
283 automated tests, with at least 80% branch coverage enforced on the core modules. State here which parts used AI assistance (FAQ 3).
	Terminal, repo history and design notes
	`$ git log --merges --format="%h %ad %s" --date=short origin/main`
	`60af190 2026-10-29 Merge remote-tracking branch 'origin/main' into fix/exact-address-search`
	`ea13a3d 2026-10-21 Merge pull request #4 from sc2006lab/lab-4-updates-ham`
	`8228cba 2026-10-21 Merge pull request #3 from sc2006lab/lab-3-updates-ham`
	`929713f 2026-09-25 Merge pull request #2 from sc2006lab/saas-platform`
	`1a1dd04 2026-09-25 Merge pull request #1 from sc2006lab/prototype`
	`$ ls prototype/specs`
	`exact-address-search_design.md exact-address-search_verification.md`
	`family-planning-v1-1_design.md family-planning-v1-1_verification.md live-data_design.md`
	`live-data_verification.md overpass-recovery_design.md reference-ui_design.md`
	`reference-ui_verification.md singapore-theme_design.md vercel-postal_design.md`
	`vercel-postal_verification.md`
	Repo history and design notes on 31 Oct

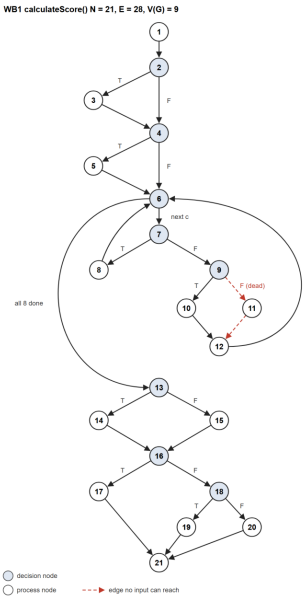
Script	Visual reference

SHOW
Slide: future upgrades.
SAY
A new data source is one adapter, one daily job and a category label. Retuning the score is a new configuration version.
POINT TO ADD
When we narrowed scope, billing and workspaces came out cleanly, with a migration that stops rather than lose data.
	prototype/server/src/synchronisation/sources.ts
	`// Data Source Registry configuration (NFR-MAINT-01): each configured public source is an isolated adapter.`
	`// Adding or replacing a source only touches this file.`
	`id: 'lta-mrt-exits',`
	`name: 'LTA MRT Station Exits',`
	`provider: 'Land Transport Authority via data.gov.sg',`
	`datasetName: 'LTA MRT Station Exit (GEOJSON)',`
	`licence: SGOL,`
	`freshnessThresholdDays: 31,`
	`minRecords: 100,`
	`categories: ['mrt'],`
	`async fetch() {`
	`export const SOURCES: SourceDefinition[] = [`
	`uraSubzones,`
	`ecdaPreschools,`
	`ecdaChildcare,`
	`moeSchools,`
	`osmSupermarkets,`
	`mohChasClinics,`
	`nparksParks,`
	`ltaMrt,`
	`ltaBusStops,`
	`];`
	Real code: sources.ts

Ilham, 16:30–19:00: traceability for UC-6.1

Script	Visual reference
SHOW Use case: UC-6.1 on the diagram. **SAY** The score is included by UC-4.1 and UC-6.2, so one class serves both.	 Use Case Descriptions v1.0, Figure 1 (crop)
SHOW Requirements: FR-DEC-01 to FR-DEC-03. **SAY** Score from weights and normalisation rules, itemised contributions, and missing data disclosed.	FR-DEC-01 Must Visitor — The system shall calculate a family suitability score for each neighbourhood using the selected criterion weights and disclosed normalisation rules. — Input: Neighbourhood evidence and criterion weights Output: Total score on a 0 to 100 scale — Compare representative and boundary results with independently calculated fixtures. FR-DEC-02 Must Visitor — The system shall display each criterion's value, normalised value, weight, and contribution so the contributions reconcile with a total rounded to the nearest whole number, with an exact 0.5 rounded upward. — Input: Score calculation Output: Itemised explanation and total — Recalculate typical, boundary, all-zero, and exact-0.5 cases. FR-DEC-03 Must Visitor — The system shall identify every criterion lacking sufficient data and disclose how its weight was treated in the score. — Input: Neighbourhood with missing criterion data Output: Missing-data label and adjusted-weight explanation — Remove each criterion in turn and verify the disclosed rule and resulting total. SRS v1.5 §4.6 (crop)
SHOW Design: SRS Table 9.5.2 and §9.4. **SAY** ScoringController, with NormalisationRule as the Strategy.	**Table 9.5.2 Control classes** Design class / Module (server/src/) / Key functions / endpoints AccountController — `account/routes.ts`, `passwords.ts` — POST /auth/register, /auth/login, /auth/logout, /auth/forgot-password, /auth/reset-password; PATCH, DELETE /account; Argon2id hashPassword() PreferenceController — `preferences/routes.ts`, `presets.ts` — GET, PUT /preferences; POST /preferences/suggest-weights; suggestWeights() DiscoveryController — `neighbourhoods/service.ts`, `routes.ts` — GET /neighbourhoods – searchNeighbourhoods(), nameMatches() NeighbourhoodController — `neighbourhoods/service.ts`, `routes.ts` — GET /neighbourhoods/:id, /:id/facilities – neighbourhoodDetail(), amenitySummaries(), facilitiesNear() RouteController — `travel/route.ts`, `neighbourhoods/routes.ts` — GET /neighbourhoods/:id/route – routeEstimate(); GET /geocode ComparisonController — `neighbourhoods/routes.ts` — GET /compare (compareQuerySchema: 2–4 ids) – summarise() per item ScoringController — `scoring/score.ts`, `config.ts` — calculateScore(evidence, weights, datasetVersions), normalise(), roundHalfUp() ShortlistController — `shortlist/routes.ts` — GET, POST /shortlist; DELETE /shortlist/:id; PUT /shortlist/:id/note SRS v1.5 Table 9.5.2 (crop)

SHOW
Code: `server/src/scoring/score.ts`, `calculateScore()` and `assertValidScoringInput()`.
SAY
The input guard exists because black-box test design showed the class relied on the API to reject bad weights.
	prototype/server/src/scoring/score.ts
	`export function assertValidScoringInput(evidence: ScoringEvidence, weights: CriterionWeights): void {`
	`const problems: string[] = [];`
	`for (const c of CRITERIA) {`
	`const w = weights[c];`
	`if (!Number.isInteger(w) || w < WEIGHT_MIN || w > WEIGHT_MAX)`
	`problems.push(`${c}: weight must be a whole number from ${WEIGHT_MIN} to ${WEIGHT_MAX} (got ${w})`);`
	`const v = evidence[c].value;`
	`if (v === null) continue;`
	`if (!Number.isFinite(v) || v < 0) problems.push(`${c}: value must be a finite number ≥ 0 (got ${v})`);`
	`else if (SCORING_CONFIG.criteria[c].kind === 'count' && !Number.isInteger(v))`
	`problems.push(`${c}: facility count must be a whole number (got ${v})`);`
	`}`
	`if (problems.length > 0) throw new ScoringInputError(problems.join('; '));`
	`}`
	`export function calculateScore(`
	`evidence: ScoringEvidence,`
	`weights: CriterionWeights,`
	`datasetVersions: Record<string, number>,`
	`config: ScoringConfiguration = SCORING_CONFIG,`
	`now: Date = new Date(),`
	`): FamilyScoreDto {`
	`assertValidScoringInput(evidence, weights);`
	`const available = CRITERIA.filter((c) => evidence[c].value !== null);`
	Real code: score.ts

Script	Visual reference
SHOW Black-box tests: test document §2. **SAY** 42 cases from equivalence classes and boundary values, such as 149, 150 and 151 metres.	**2.4 Test cases and results** Inputs not listed are held at their nominal value (every criterion normalises to 0.5). In the D, C, M and R groups only the criterion under test is weighted (weight 5). In the W group, childcare = 30 (normalised 1) and schools = 0 (normalised 0) with schools weight 5, so the total is w ÷ (w + 5) × 100. Values in brackets are the unrounded totals. ID | Test input | Class / boundary | Expected output | Actual output | Result BB-D1 | groceries = −1 m | EC-D1; BV below 0 | ScoringInputError | ScoringInputError thrown | Pass BB-D2 | groceries = 0 m | EC-D2; BV 0 | total 100 | total 100 | Pass BB-D3 | groceries = 149 m | EC-D2; BV 150−1 | total 100 | total 100 | Pass BB-D4 | groceries = 150 m | EC-D2; BV 150 | total 100 | total 100 | Pass BB-D5 | groceries = 151 m | EC-D3; BV 150+1 | total 100; normalised 0.999048 | total 100; normalised 0.999048 | Pass BB-D6 | groceries = 675 m | EC-D3; nominal | total 50 | total 50 | Pass BB-D7 | groceries = 1199 m | EC-D3; BV 1200−1 | total 0; normalised 0.000952 | total 0; normalised 0.000952 | Pass BB-D8 | groceries = 1200 m | EC-D4; BV 1200 | total 0 | total 0 | Pass BB-D9 | groceries = 1201 m | EC-D4; BV 1200+1 | total 0 | total 0 | Pass Test document v1.0 §2.4 (crop)
SHOW White-box tests: test document §3. **SAY** 9 basis-path cases. Cyclomatic complexity is 9 for `calculateScore()` and 5 for `dedupeFacilities()`.	 Test document v1.0, Figure 1

SHOW
Live run: `npx vitest run test/scoring.blackbox.test.ts test/whitebox.test.ts` in `prototype/server`.
SAY
Both files run without a database, so this is safe to run live.
	Terminal, run on 31 Oct 2026
	$ cd prototype/server
	$ npx vitest run test/scoring.blackbox.test.ts test/whitebox.test.ts
	RUN v5.0.2 prototype/server
	Test Files 2 passed (2)
	Tests 52 passed (52)
	Start at 17:45:01
	Duration 382ms (import 55%, transform 30%, tests 12%, worker 3%)
	Real run on 31 Oct: 52 tests passed

Apollo, 19:00–19:30: close

On the landing page: "The Tans went from separate sites to one shortlist they understand. FamPlan runs on public data that refreshes itself, a score that explains itself, and a design that makes new sources cheap to add."

The old script's Q&A and failure-plan tables are still useful once the answers are reassigned to this presenter list and corrected with the claims table below.

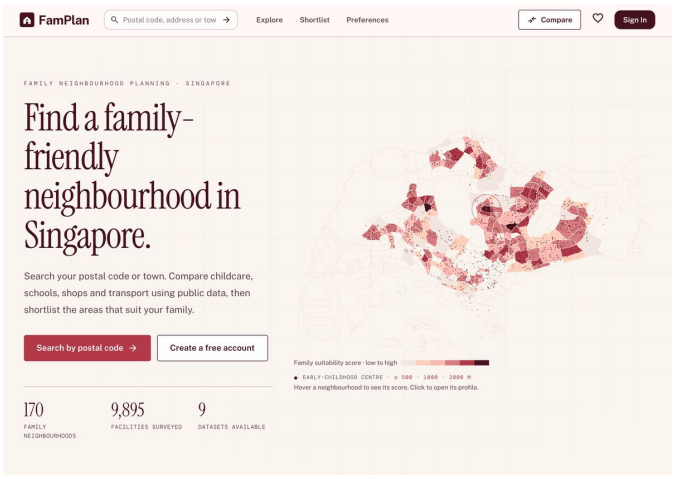
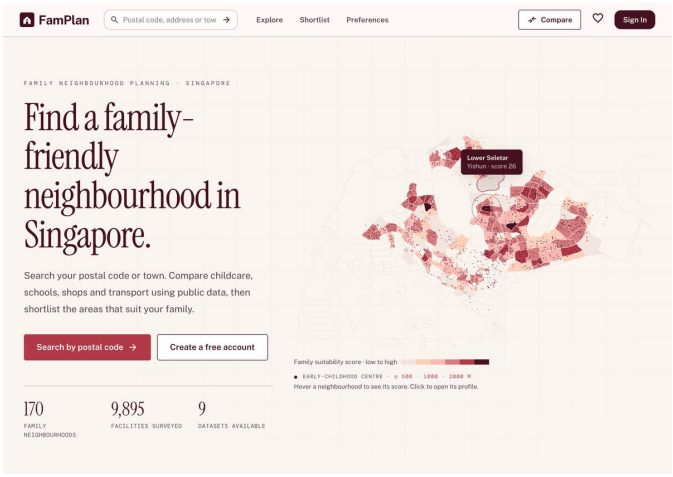
Demo video script

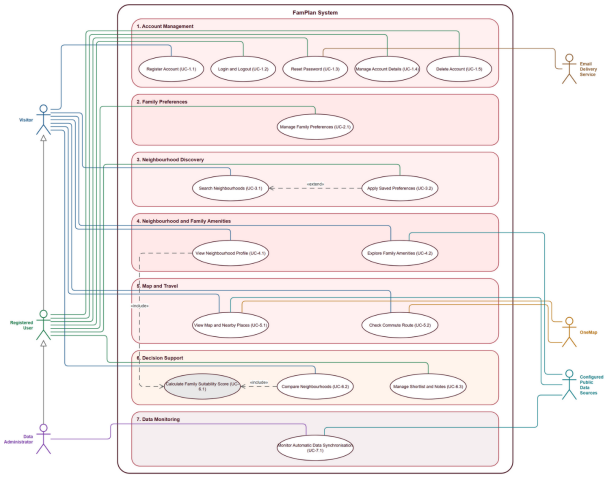
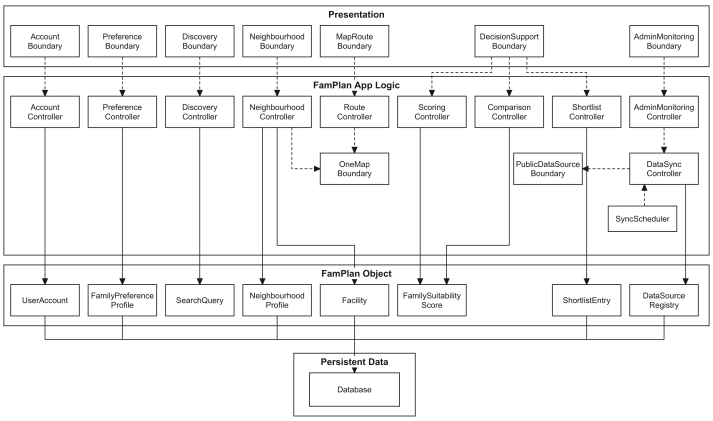
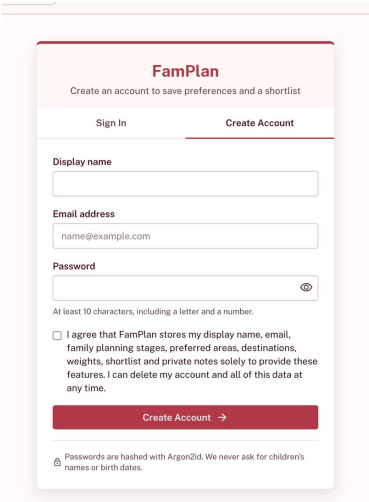
This is the full voice-over, word for word, for one narrator. Each row is one clip, so you can record screen and voice separately and retake a single clip. It meets the 5 to 7 minutes in Lab 5 §3.3. At an easy reading pace it runs about 6:50. If the TA holds us to the five minutes in Lab 4 §3.4.4, drop the clips marked "can be cut" to reach about 6:10. Keep clip 4, because the teacher asked for the architecture near the start.

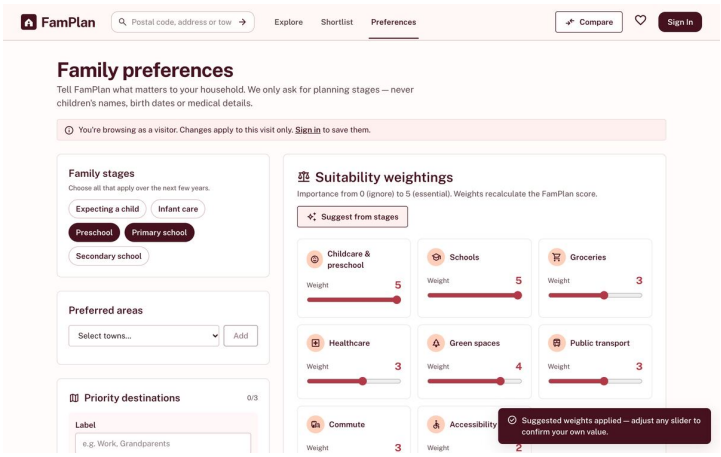
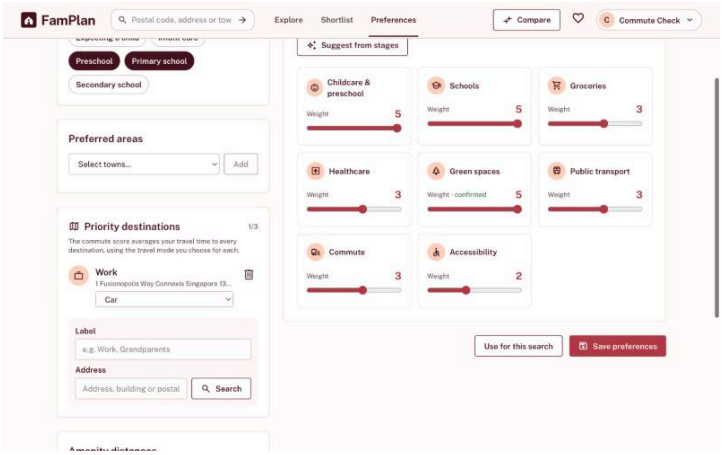
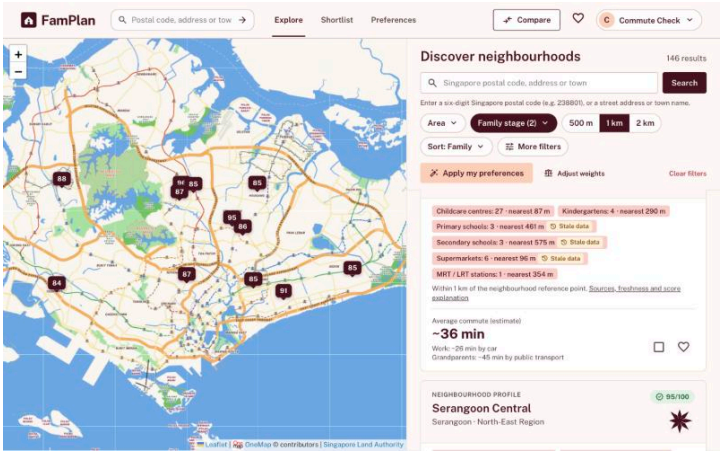
Before recording

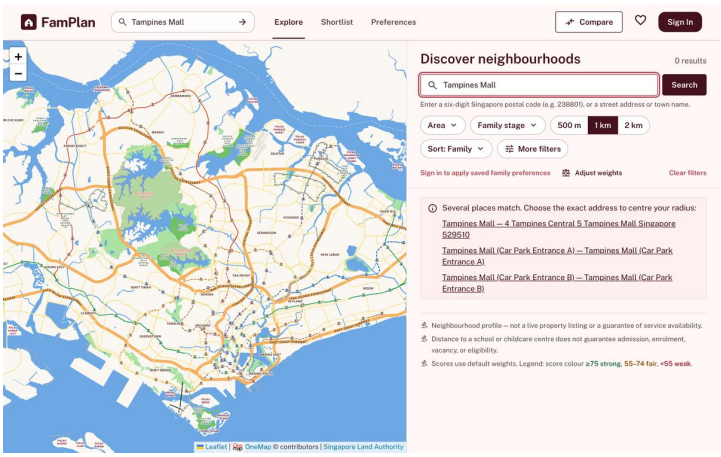
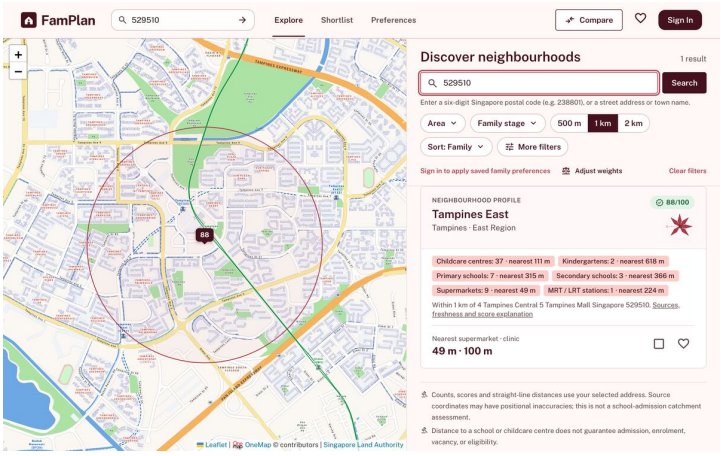
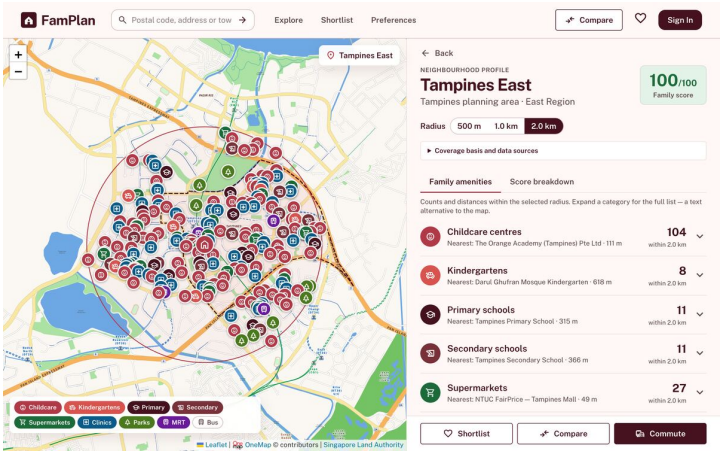
- ☐ **OneMap:** on the live site, run a postal search for 138632 and a commute route. If either fails or shows Estimate, sort out the OneMap login first. If the site uses a static token, record before Sunday 1 Nov at 18:04.
- ☐ **Administrator login:** confirm one works on the live site. Otherwise record clips 20 to 22 from a local copy and say so on screen.
- ☐ **Registration:** use a fresh email alias, and pause recording while typing the password. Never show passwords, or the Vercel or OneMap dashboards.
- ☐ **Windows:** one signed-out window, one for the new registered account, and one for the administrator. Load Explore and a profile first, because the first call after idle is slow.
- ☐ **Screen:** 1920 × 1080, browser zoom about 110%, bookmarks bar hidden, notifications off.
- ☐ **Commute update:** record after Edward deploys it, because both scripts now show travel modes and two destinations. Until then, the live site shows the old labels and scores only the first destination.
- ☐ **AI disclosure:** decide whether to add a closing line on AI assistance (FAQ 3 and 4). It is not in the script below.
- ☐ **Submit:** export 1080p. If it is too large for the repo, upload to YouTube as unlisted and put the link in the index file (Lab 5 §3.3.3). Keep a local copy as the live-demo backup.

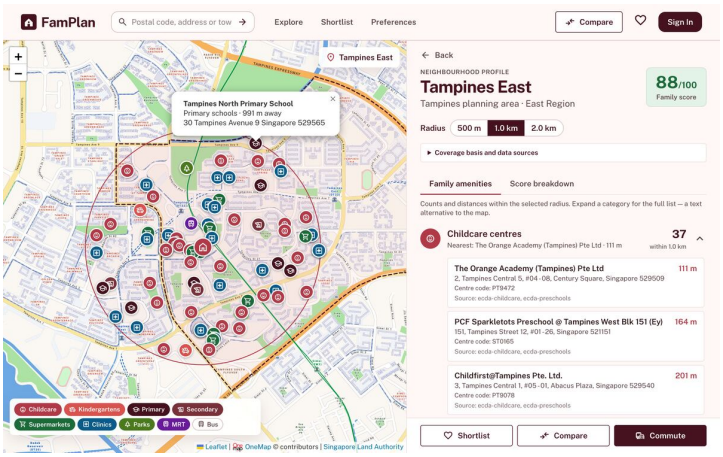
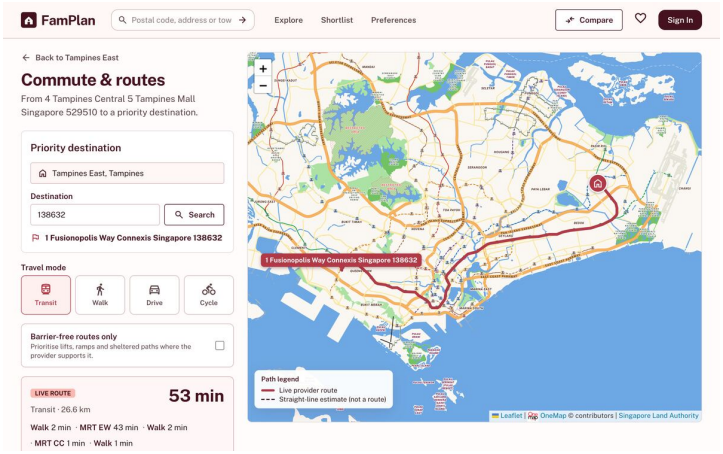
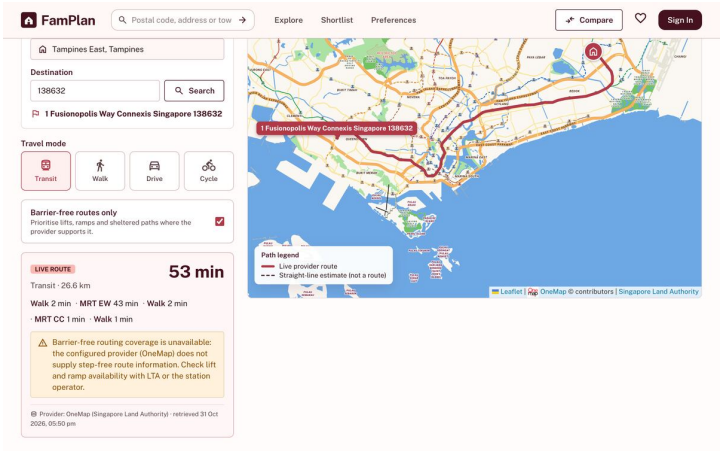
Voice-over, clip by clip

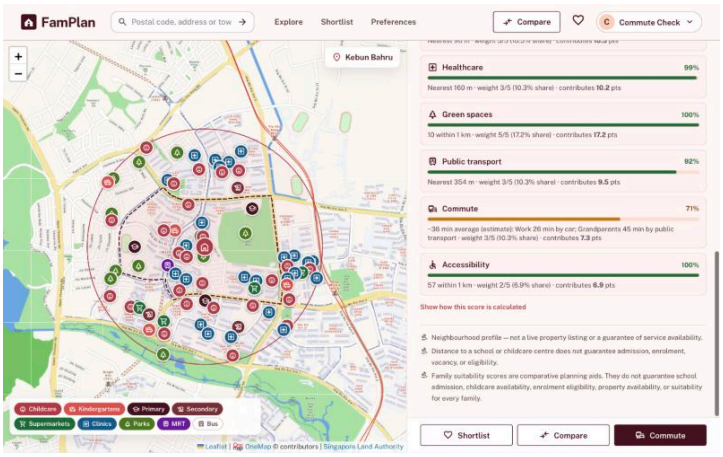
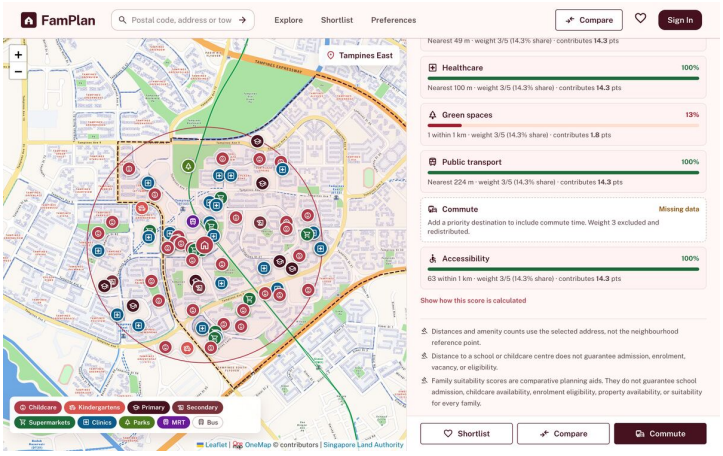
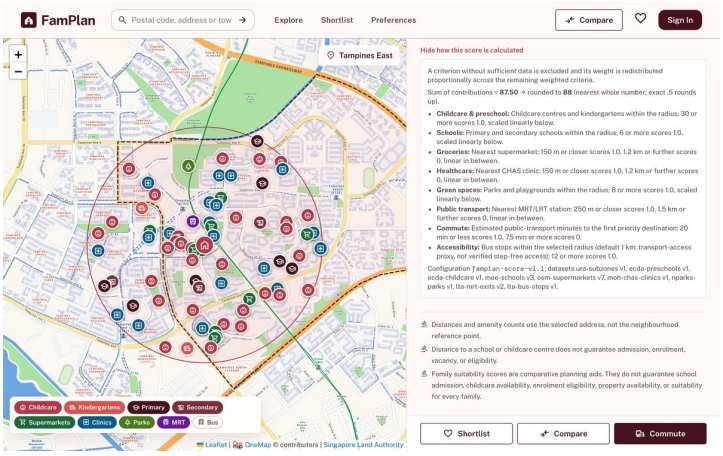
Script	Visual reference
Clip 1 · 0:00–0:25 **ON SCREEN** Landing page. Scroll slowly until the counters run. **VOICE-OVER** Choosing where to raise a family is one of the biggest decisions a couple makes in Singapore. You end up checking childcare, schools, clinics, supermarkets, MRT and commute times across separate websites. FamPlan brings all of that into one place, and gives each neighbourhood a family score you can actually explain.	 Live site
Clip 2 · 0:25–0:36 **ON SCREEN** Hover a few neighbourhoods on the map. **VOICE-OVER** It covers 170 family neighbourhoods, built from nine public datasets, including ECDA, MOE, MOH, NParks and LTA.	 Live site

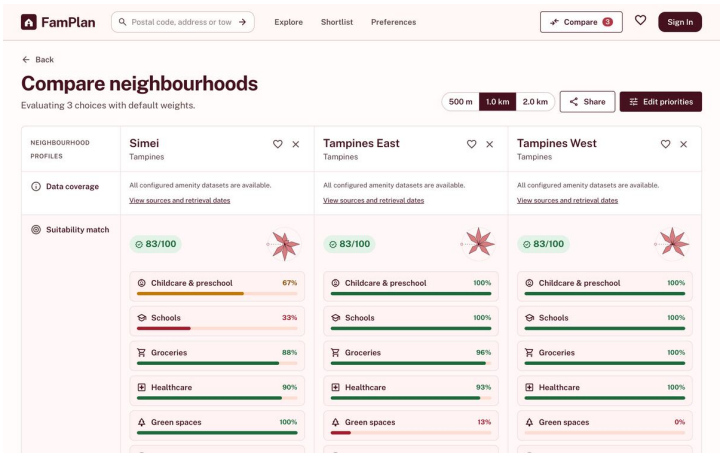
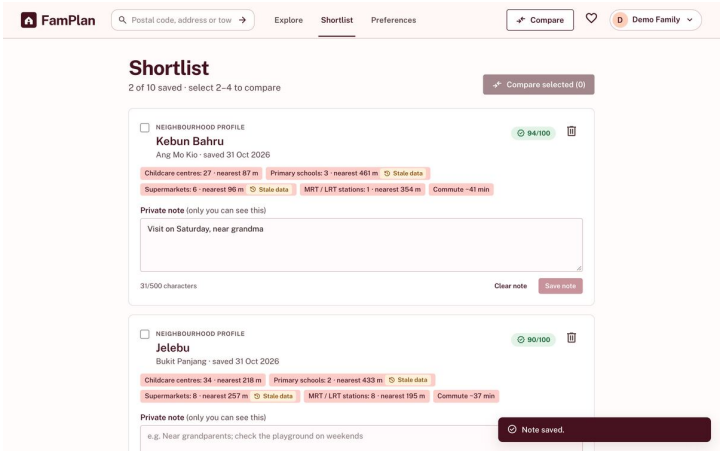
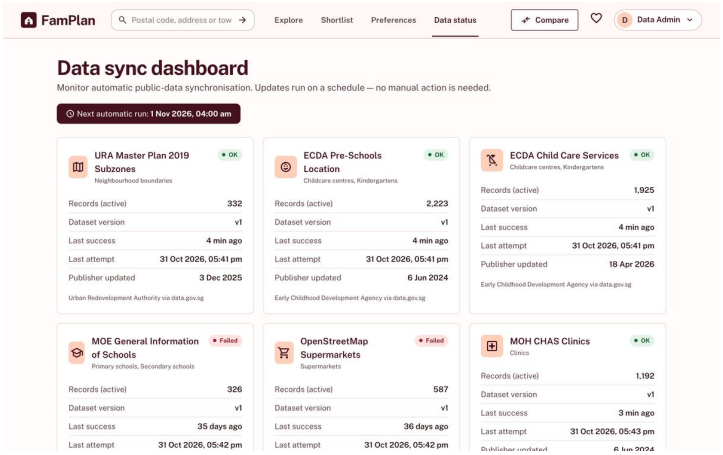
Script	Visual reference
Clip 3 · 0:36–0:59 **ON SCREEN** Use case diagram, full screen. **VOICE-OVER** FamPlan has three kinds of user. A visitor can search, view profiles, compare neighbourhoods and check commutes without an account. A registered user can also save preferences, a shortlist and private notes. A data administrator monitors the automatic data updates. That makes 16 use cases in total.	 Use Case Descriptions v1.0, Figure 1
Clip 4 · 0:59–1:22 **ON SCREEN** Architecture figure, full screen. **VOICE-OVER** Under the hood, it has a React and TypeScript front end, an Express API and PostgreSQL through Prisma, with Leaflet and OneMap for maps. It is deployed on Vercel in Singapore, and in a local load test one server handled 1,000 simulated users with no failed requests.	 SRS v1.6 §8.2
Clip 5 · 1:22–1:45 **ON SCREEN** Register page. Fill in the name and email, cut while typing the password, tick consent, submit. **VOICE-OVER** Let's follow one family, the Tans. They have a toddler, a child starting Primary 1, and a parent who works at one-north. They create an account. The password needs at least 10 characters, and they agree to how their data is used before anything is stored.	 Live site

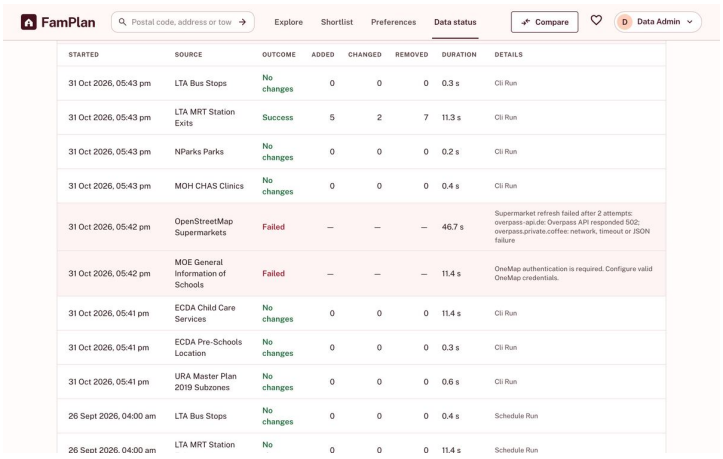
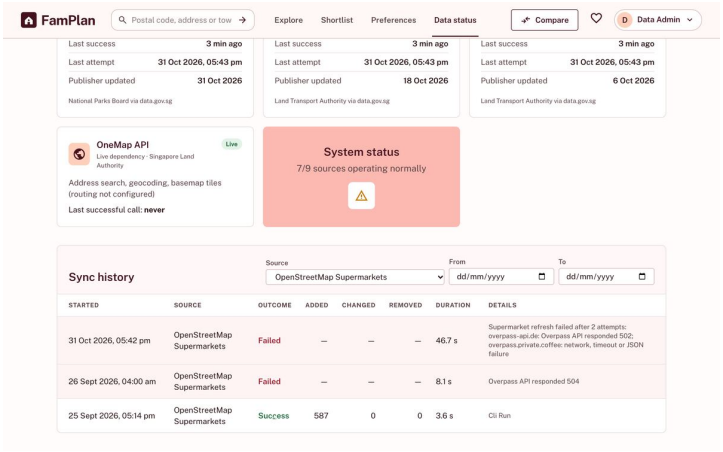
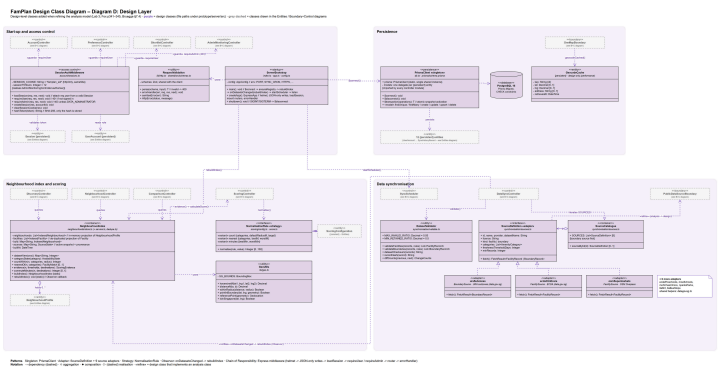
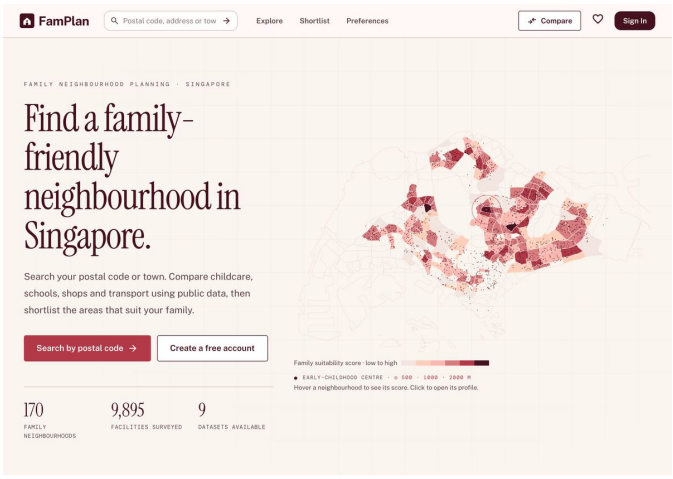
Script	Visual reference
Clip 6 · 1:45–2:05 **ON SCREEN** Preferences. Tick Preschool and Primary school, click Suggest from stages. **VOICE-OVER** Next, they tell FamPlan what matters to them. They pick the Preschool and Primary school stages, and FamPlan suggests weights. Zero means ignore, five means essential. FamPlan only asks for planning stages, never children's names, birth dates or medical details.	 Live site, visitor view. A registered account sees Save preferences.
Clip 7 · 2:05–2:23 **ON SCREEN** Move Green spaces to 5. Add the label Work and postal code 138632, choose Car, and click Save preferences. **VOICE-OVER** They move a slider to set their own priority, and that choice is kept even if they ask for suggestions again. Then they add one-north as a destination, using its postal code, choose how they get there, and save.	 Local test copy, signed in
Clip 8 · 2:23–2:47 **ON SCREEN** Explore. Click Apply my preferences. In Preferences, add Grandparents at Bedok MRT by Public transport and save. In Explore, apply again. **VOICE-OVER** On the Explore page, Apply my preferences ranks neighbourhoods by this family's weights, and narrows the list to their family stages. They also visit grandparents in Bedok by public transport, so they add that too. The commute score averages both trips, and Katong moves into their top three.	 Local test copy, signed in

Script	Visual reference
Clip 9 · 2:47–3:06 **ON SCREEN** Search 529510, then search Tampines Mall and show the three matches. **VOICE-OVER** Families think in addresses, not planning zones, so we added exact search. A postal code goes straight to that address. A place name works too. Tampines Mall matches three places, so FamPlan asks which one instead of guessing.	 Live site
Clip 10 · 3:06–3:20 **ON SCREEN** Pick the first match. Pause on the radius circle and the notice. **VOICE-OVER** Once they choose, OneMap resolves the coordinate, and the radius, counts and score all start from that address, not the centre of the zone.	 Live site
Clip 11 · 3:20–3:41 **ON SCREEN** Open the profile. Click 500 m, 1 km, then 2 km. **VOICE-OVER** This is the profile for that address. Switching the radius from 500 metres to one kilometre to two kilometres updates the map circle and every count together. These are straight-line distances, so FamPlan does not present them as walking routes or school catchments.	 Live site, 2 km

Script	Visual reference
Clip 12 · 3:41–3:59 · can be cut **ON SCREEN** Toggle a map layer, click a marker, expand Childcare centres. **VOICE-OVER** The map layers show each type of facility. Click a marker for its details, or expand a category for a text list, which is an accessible alternative to the map. Each entry names its data source.	 Live site
Clip 13 · 3:59–4:15 **ON SCREEN** Commute page. Saved destination Work opens on Car. Click Public transport, Walk, Cycle. **VOICE-OVER** Now the commute. It opens on one-north by car, the mode they saved, with a live OneMap route showing each leg. They can preview public transport, walking and cycling too.	 Live site
Clip 14 · 4:15–4:31 **ON SCREEN** Back to Public transport. Tick Barrier-free routes only and pause on the notice. **VOICE-OVER** If they ask for barrier-free routes, FamPlan says that OneMap does not supply step-free data, rather than guessing. And if routing is ever unavailable, it shows a clearly labelled estimate.	 Live site

Script	Visual reference
Clip 15 · 4:31–4:44 **ON SCREEN** Profile. Score breakdown tab. Scroll through the rows. **VOICE-OVER** Here is what makes FamPlan different. The score breakdown shows each criterion's raw value, its normalised value, its weight and what it contributes.	 Local test copy, signed in, two destinations
Clip 16 · 4:44–4:55 **ON SCREEN** Signed-out window, same profile. Pause on the Commute row. **VOICE-OVER** If data is missing, FamPlan says so. Here, without a destination, commute is excluded and its weight is redistributed.	 Live site, signed out
Clip 17 · 4:55–5:14 **ON SCREEN** Click Show how this score is calculated. Pause on the configuration line. **VOICE-OVER** The rules sit in one versioned configuration, so the same inputs and the same data always give the same score. And the score is a comparison aid. It never promises a school place or a childcare vacancy.	 Live site

Script	Visual reference
Clip 18 · 5:14–5:25 **ON SCREEN** Add three neighbourhoods to Compare and open it. **VOICE-OVER** The Tans pick three neighbourhoods and compare them side by side. FamPlan can compare up to four.	 Live site, default weights
Clip 19 · 5:25–5:38 **ON SCREEN** Save two, open Shortlist, type a note and save it. **VOICE-OVER** They save two to their shortlist and add a private note. The shortlist holds up to 10 neighbourhoods, each with its own note.	 Local test copy, signed in
Clip 20 · 5:38–5:53 **ON SCREEN** Administrator window. Data sync dashboard, scroll the source cards. **VOICE-OVER** Finally, the data administrator. There is no upload button here. All nine datasets refresh automatically every day, and each one shows its status, last success and record count.	 Local test copy. Live will differ.

Script	Visual reference
Clip 21 · 5:53–6:05 **ON SCREEN** Scroll to Sync history. **VOICE-OVER** Every run is logged, with the records added, changed and removed. New data must pass validation before it goes live.	 Local test copy
Clip 22 · 6:05–6:16 **ON SCREEN** Filter by source. Pause on a Failed row if there is one, or on the source card if not. **VOICE-OVER** If a provider fails, the last validated data stays active, so families never see an empty map.	 Local test copy, filtered to supermarkets
Clip 23 · 6:16–6:37 · can be cut **ON SCREEN** Design layer figure, full screen. **VOICE-OVER** This comes from the design. Each data source sits behind its own adapter, and the scoring rules live in one versioned configuration, so adding a source or retuning the score stays a small change. More than 280 automated tests back this up.	 SRS v1.6 §9.4
Clip 24 · 6:37–6:50 **ON SCREEN** Back to the landing page. **VOICE-OVER** So the Tans went from separate websites to one shortlist they understand. That is FamPlan, from team Banana Five-O. Thanks for watching.	 Live site

Claims to use and drop

These were checked against the current code, a clean test run and the live site on 31 Oct.

Claim	Status	What to say instead
Handles 1000+ users	**Local only**	Say: "In a local load test, one server handled 1,000 simulated users browsing with no failed requests and a p95 of about 1.3 seconds." It was read-only and not on Vercel, so do not say the live site handles 1,000+ users. Evidence is in the test document §4.3.
Sync at 4 am daily	**Out of date**	On Vercel, each source runs once a day in its own overnight window, and timing is approximate.
Stale after 24 hours	**Wrong**	Stale after 31 days. Delayed after 26 hours.
famplan-score-v1.0	**Out of date**	famplan-score-v1.2, effective 31 Oct.
Team workspaces and paid plans show extensibility	**Removed**	Delete this line. The code is gone. Use the scope-change story instead.
Docker deployment	**Out of date**	Vercel and Neon PostgreSQL. Docker is only an optional local setup.
170 tests, all passing	**Out of date**	283 tests pass: 255 server and 28 client, on 31 Oct.
UC-6.1 is included by Search	**Wrong**	The use case document says it is included by UC-4.1 and UC-6.2.
Every marker shows its source and date	**Overstated**	List entries name their source. Dates are under Coverage basis and data sources. The marker popup shows name, distance and address.
Verified accessible routes	**Overstated**	OneMap supplies no step-free data, and the accessibility score is a bus-stop proxy. The app says so.
Travel modes and multiple commute destinations	**Confirmed**	On main, tested, and checked in the app on a local copy on 31 Oct. Not on the live site until the update is deployed.
9 data sources	**Confirmed**	9 adapters with 9 daily jobs. OneMap is a separate live service on top.
170 neighbourhoods	**Confirmed**	Live site total. The landing page also shows 9,895 facilities.
16 use cases	**Confirmed**	Use Case Descriptions v1.0.
Data refreshes automatically	**Confirmed**	All 9 sources were retrieved between 01:50 and 08:47 Singapore time on 31 Oct.
At least 80% branch coverage	**Confirmed**	Enforced per file for the NFR-MAINT-02 modules. The coverage run passes at 93.88% overall.
Live routes in 4 modes	**Confirmed**	Public transport, walk, car and cycle all returned OneMap routes on 31 Oct. The live site labels them Transit and Drive until the update is deployed.

Fix before the demo

Both commute items are fixed on main and reach the live site after the deploy below. Registration, saved preferences, the shortlist and the admin pages were not run on the live site, because they need accounts. All of them ran on a local copy, so the live paths still need a run-through.

- ☐ **Deploy the commute update.** Edward has the Vercel account. Run npm run db:deploy against the production database first, then deploy from prototype/. Deploying without the migration breaks signed-in preferences.
- ☐ **Commute mode in preferences: done.** Each destination saves a travel mode, and search and the score use it. The score version is now famplan-score-v1.2.
- ☐ **Multiple destinations in ranking: done.** The commute score, sort and filter use the average across all saved destinations, with equal weight.
- ☐ **OneMap login on Vercel.** Postal search and routing work today. A spec note says the team's renewed token expires on 1 Nov at 18:04 Singapore time, and static tokens last only 3 days. Whoever owns the Vercel project should confirm `ONEMAP_EMAIL` and `ONEMAP_PASSWORD` are set. The project was not visible from the Vercel account used for this check.

- ☐ **Live accounts.** The README says never to run the demo seed on production, so `demo@famplan.sg` and `admin@famplan.sg` may not exist there. Edward should confirm an administrator login, and we should set up a prepared registered account.
- ☐ **Failed sync row.** Do not force a failure on the live site. Look for a real one in Sync history. If there is none, describe it in one sentence and point to `server/test/sync.test.ts`.
- ☐ **Diagrams.** SRS v1.6 enlarges the system architecture figure on its own landscape page. Ilham is re-rendering SRS §9.2 and §9.3.
- ☐ **Traceability laptop.** In `prototype`, run `npm ci` and `npm run build -w shared` once before the demo, so the two test files run cleanly live.
- ☐ **Warm-up.** The first API call after idle took 2.8 s, against about 70 ms when warm. Load Explore and one profile 5 minutes before starting.
- ☐ **Backups.** The old script's `Lab4/Prototype_Screenshots` folder is not in the repo. Use the screenshots in this document or drop that line, and keep the video saved locally.
- ☐ **Rehearsal.** Run the full 20 minutes twice with a timer, once switching to the backup video.

Lab 4 checklist

Lab 4 is due before the Lab 5 session starts, when its marks lock (FAQ 9). The AI report is the largest piece left.

Deliverable	Manual	Status	What is left
AI report (PDF)	§3.3.4, §4	**Not in repo**	Not on any branch. Run both experiments and write both reflections (steps below).
Demo video	§3.4.4	**Not in repo**	Script is in this document. Record 5 to 7 minutes this weekend. That also meets Lab 5 §3.3 and gives us the live-demo backup.
Demo script	§3.4, §4	**Needs update**	This outline is updated for the commute work, and a PDF is in Lab4/. Add authors and re-export.
Test cases and results	§3.2, §4	**Done**	v1.1 is in Lab4/. Re-run on 31 Oct with 283 tests, and adds commute tests and a local load test. It notes that the two scoring test files run 52 tests against the 51 cases listed.
Working prototype	§4	**Needs update**	Visitor search, profile and commute flows work on the live site. The commute gaps are closed on main, so deploy the update.
Author names	§3.5.2	**Needs update**	The test document has them. Add them to the demo script and the AI report.
AI disclosure	FAQ 3	**To confirm**	Confirm any other AI use is disclosed in SRS §1.6, and that each presenter can explain the code behind their part.
Source code	§3.1.4, §4	**Done**	On `main` under `prototype/`. Merge any late fixes.

AI report steps

1. **Rebuild the control class (§3.3.2).** On a throwaway branch, delete `server/src/scoring/score.ts`. Ask an in-IDE agent such as Cline to rebuild it from the tests, SRS and diagrams. Judge it with the 52 black-box and white-box tests. If it fails, retry with only the function bodies removed. Do not merge this branch.
2. **Run a second experiment (§3.3.3).** The commute update was built with an AI agent on 31 Oct and fits the manual's "feature you didn't have time to complete" option, so it can be written up here. Otherwise, pick another unfinished feature.
3. **Write two reflections (§3.3.4).** For each one, cover the prompts, materials supplied, any iteration, the outcome, and whether it met expectations. If the AI fell short, add what might have helped.
4. **Submit.** Combine both into one PDF with author names and put it in `Lab4/`.

Lab 5 checklist

Lab 5 documents are due at 23:59 on the Sunday of demo week. Two items were missing from the team's notes: the index file and the meeting minutes.

Documents and code

Deliverable	Manual	Status	What is left
Final SRS, including the use case model	§3.2.2	**Needs update**	v1.6 is in Lab5/. It adds travel modes, the enlarged architecture figure and the Vercel and cron wording, and removes the billing, workspace and Docker lines. Still to add: exact address search and Ilham's re-rendered figures. Lab5/FamPlan_SRS_v1.6_Word_Changes.md lists the same changes for the Word file.
Class diagrams of key classes	§3.2.2	**Needs update**	Currently SRS §9.1 to §9.4. §9.1 and §9.4 now show travel modes (Lab5/figures). Redraw to match the final code: billing and workspaces are gone, and the serverless, cron, Overpass and postal modules are new.
Dialog map	§3.2.2	**To confirm**	In SRS §10 and `Lab2/DialogMap.pdf`. Its pages line up with today's routes. Recheck the transitions for the exact-address search flow.
Source code and tests	§3.2.1	**Needs update**	`main holds the code and 283 passing tests. No final source zip exists yet. Build it after the last merge, without node_modules or .env.`
Index file	§3.2.3	**Not in repo**	One page linking every deliverable, including the video link.
Meeting minutes	§3.2.2	**Not in repo**	Submit any the team has kept.
Authors and declaration form	FAQ 12	**To do**	Name the authors on each document and fill in the form. SRS §1.6 lists Ron and Anh, while the chat table says Apollo and Raph, so confirm the official names.
Release tag and backup	Overview appendix, FAQ 13	**To do**	The repo has no tags yet. Tag the final version, and keep a backup because repo access ends at the deadline.

Demo

Deliverable	Manual	Status	What is left
Slides	§3.1.5, §3.1.6	**Not in repo**	A short deck on SE practices, system design and traceability. The Edward and Ilham tables above give the content.
Rehearsal	§3.1.1	**To do**	About 20 minutes: live demo, design overview, and traceability for UC-6.1.
Demo video	§3.3	**Not in repo**	5 to 7 minutes, uploaded or linked. Same recording as Lab 4.

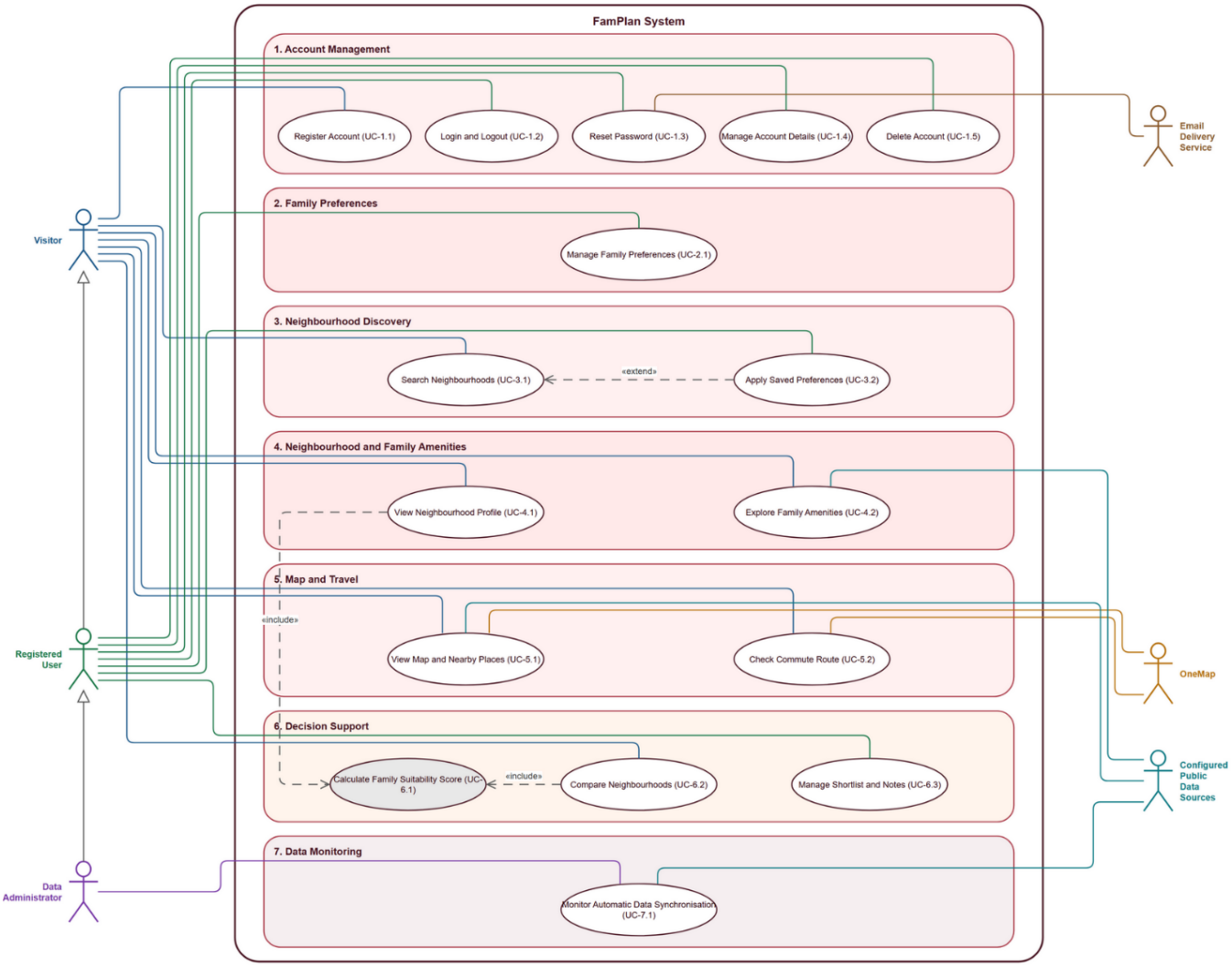
Individual

Deliverable	Manual	Status	What is left
Peer review	§3.4, FAQ 11	**To do**	Each member submits alone, without discussing it. Justify any score of 1 or 2. Details come before the Lab 5 session.

Appendix: full-size diagrams

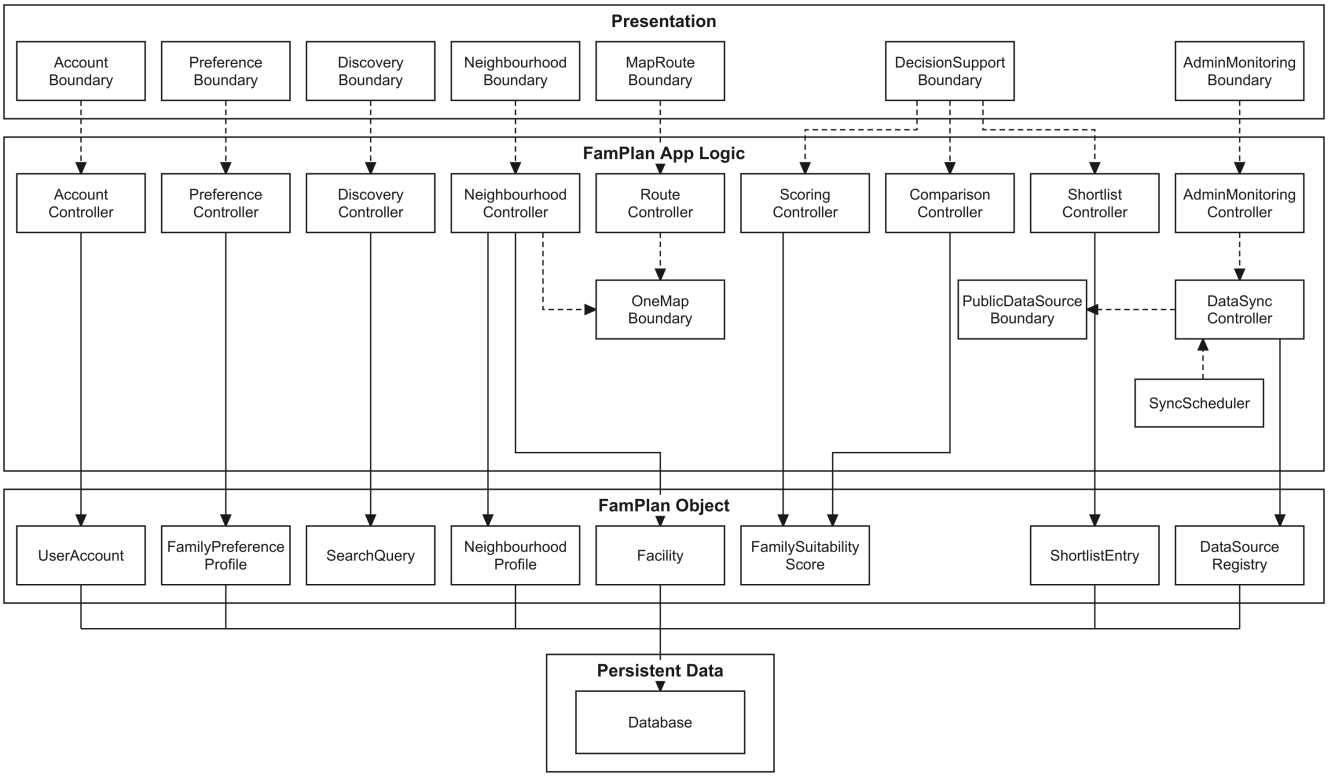
These are the current figures at page width, for slides and for spotting what needs redrawing. The system architecture and design layer figures come from SRS v1.6. The others come from the Lab 3 documents, so they predate Ilham's re-renders.

Use case diagram



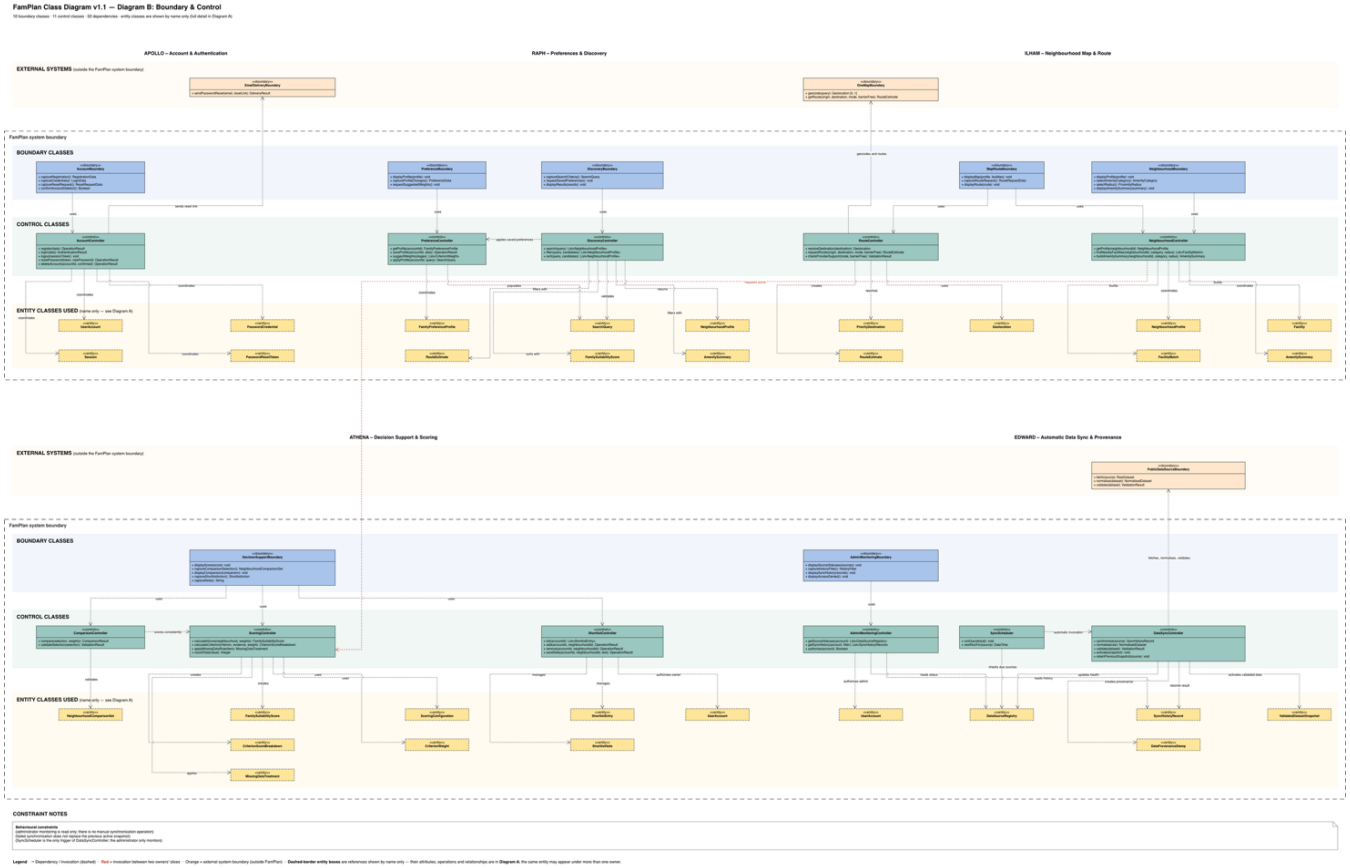
Use Case Descriptions v1.0, Figure 1

System architecture



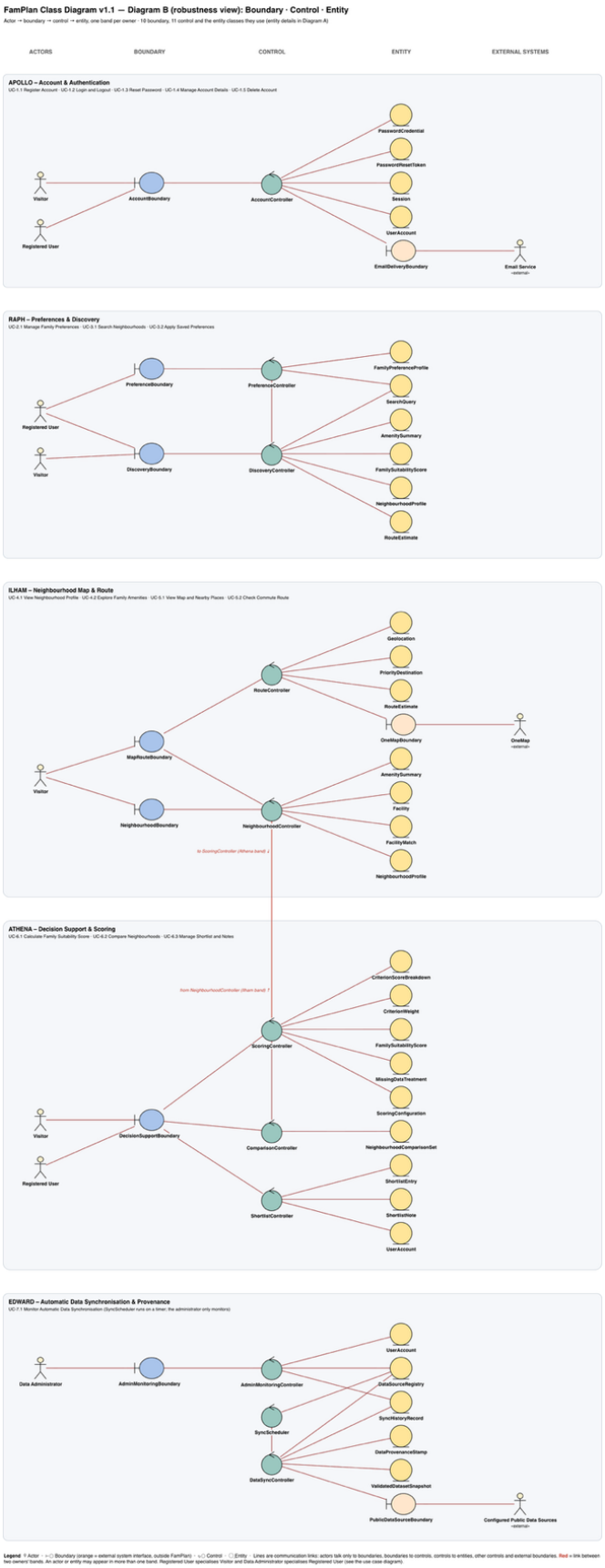
SRS v1.6 §8.2

Boundary and control subsystems



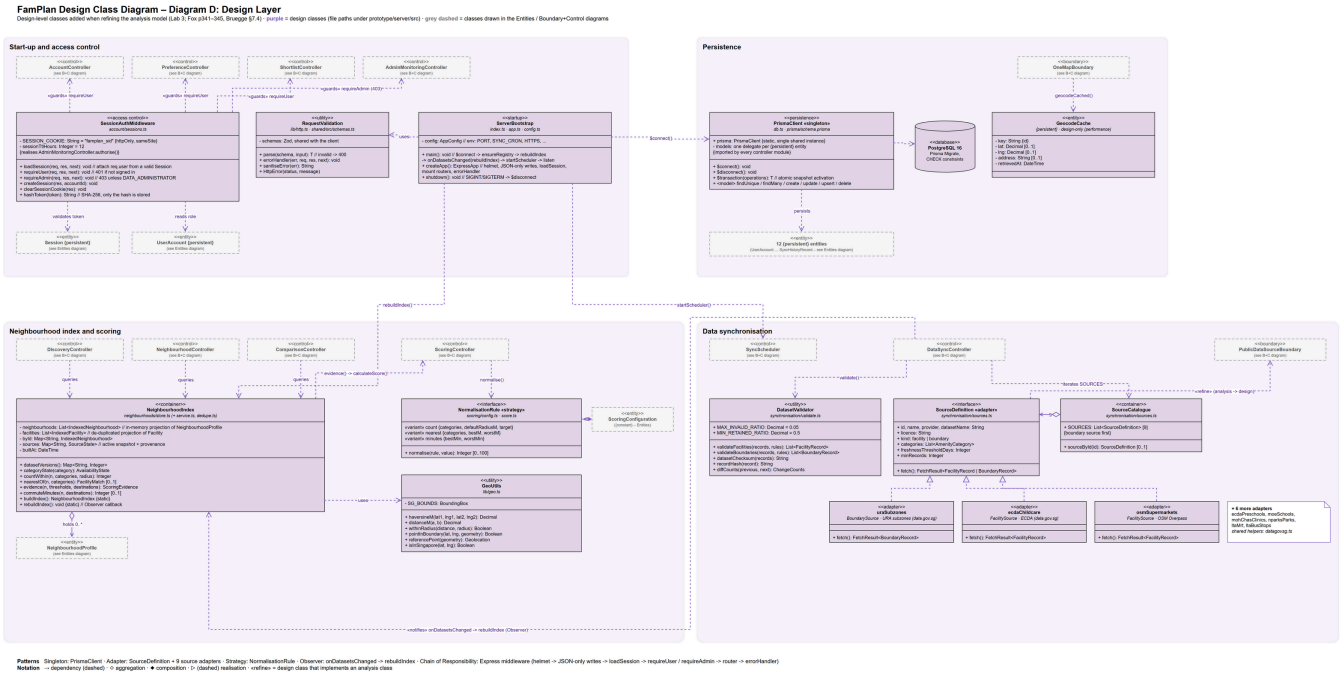
SRS v1.5 §9.2. Being re-rendered by Ilham.

Robustness view



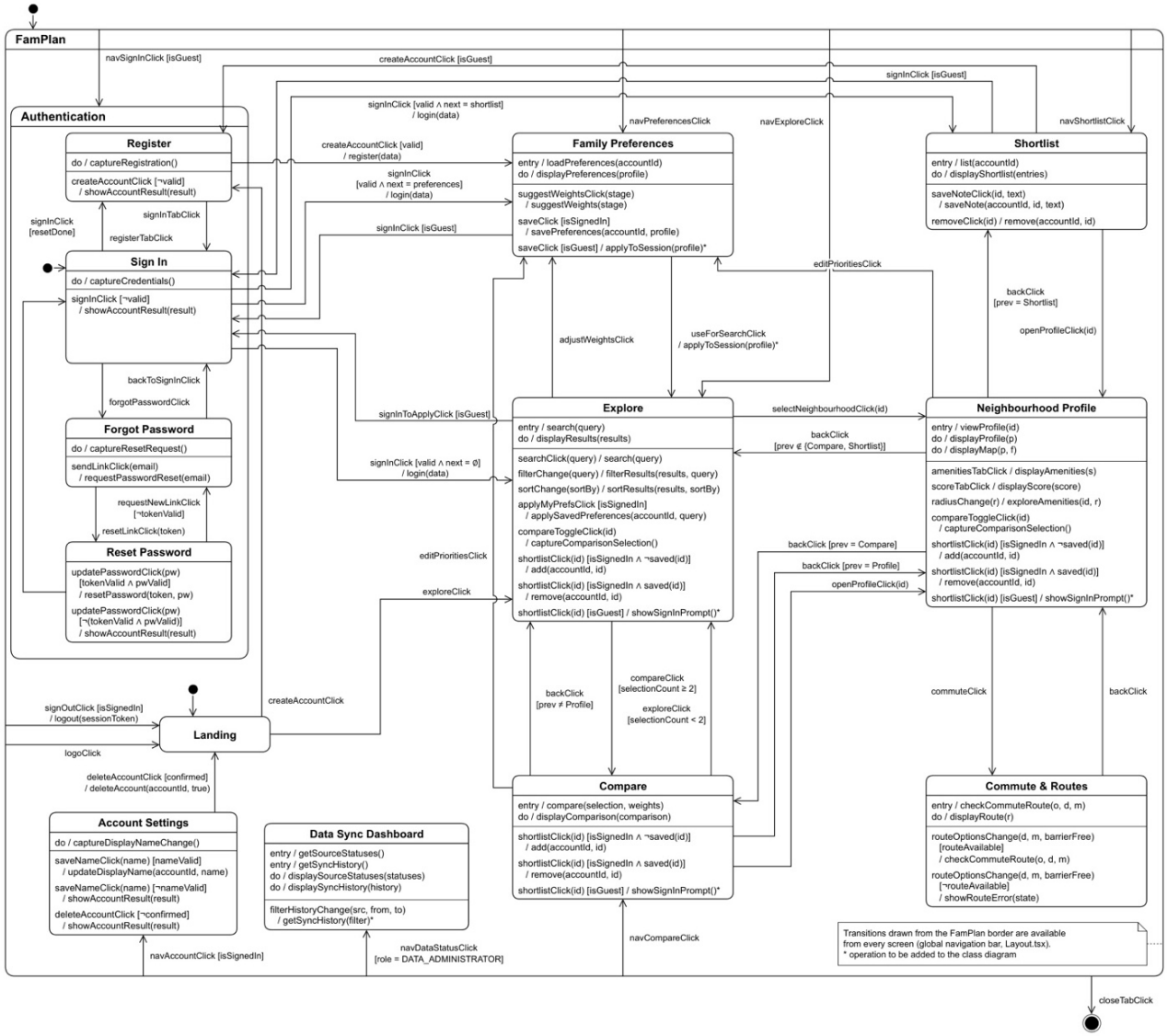
SRS v1.5 §9.3. Being re-rendered by Ilham.

Design layer



SRS v1.6 §9.4

Dialog map



SRS v1.5 §10